

Paila Nepal:
An Offline-First Rural Tourism Mobile
Application
for Gandaki Province, Nepal

Individual Honours Project Report

Abhishek Paudel
Student ID: 23140737

BSc (Hons) Computer and Data Science
Birmingham City University

Module: CMP6200
Supervisor: Sumanta Silwal
Word Count: 8,000

May 2026

Abstract

Paila Nepal is an offline-first rural tourism mobile application designed to provide destination discovery, map access, route guidance, and personalized recommendations for travelers exploring Gandaki Province, Nepal, particularly in regions with limited or unstable internet connectivity. The application addresses a critical gap in existing tourism platforms: most popular map and travel applications rely on continuous internet access and are optimised for well-known commercial destinations, leaving rural tourism locations with inadequate digital representation.

This project implements a Flutter-based Android application supported by a FastAPI backend, bundling 300 curated Gandaki Province destinations, 771 accommodation records, local photographs, descriptions, structured attributes, and precomputed semantic embeddings. The final recommendation architecture combines SBERT semantic retrieval, collaborative and popularity signals, contextual reranking, machine-learning rescoring, cold-start handling, diversity control, and explanation generation. The mobile application preserves recommendation capability offline using local embeddings and contextual scoring. Local persistence uses SQLite for saved destinations and reviews, Hive for recommendation caches, preferences, and queued interactions, and SharedPreferences for lightweight settings. A bundled MBTiles package provides offline map display. The chatbot combines local language processing, hybrid intent classification, retrieval-augmented response generation, emergency detection, dialogue memory, translation support, and optional online language-model enhancement.

Evaluation used five tourism recommendation scenarios and the ranking metrics Precision@10, Recall@10, normalised Discounted Cumulative Gain at 10, and Mean Reciprocal Rank. The hybrid recommender achieved average Precision@10 of 0.6800, Recall@10 of 0.4408, nDCG@10 of 0.7543, and MRR of 1.0000. Current automated verification also shows that Flutter static analysis passes and nine backend tests pass. The full Flutter suite exposed one remaining widget-test lifecycle issue, while a focused performance test showed that warm offline translation exceeded its 500 ms target on the test machine. These findings support the viability of the architecture while identifying concrete limitations for further improvement.

Paila Nepal demonstrates how offline-first mobile architecture, combined with curated rural tourism content, can improve information accessibility and travel safety for rural

tourism in Nepal. The project achieves a balance between comprehensive feature richness and practical offline reliability, contributing to both academic understanding of offline-first design patterns and practical tools for rural tourism promotion.

Keywords: rural tourism, Flutter, offline-first architecture, Gandaki Province, MBTiles, OSRM routing, SQLite, recommendation systems, chatbot, OpenStreetMap, mobile application design.

Acknowledgements

I would like to express my sincere gratitude to Sumanta Silwal, my supervisor at Birmingham City University, for his guidance, constructive feedback, and encouragement throughout this project. His insights on system design, offline-first architecture, and mobile application development have been invaluable.

I am grateful to the Faculty of Computing, Engineering and the Built Environment at Birmingham City University for providing a rigorous academic environment and access to resources necessary for this work. The course CMP6200: Individual Honours Project has allowed me to integrate knowledge from multiple domains—software engineering, data structures, mobile development, and tourism informatics—into one cohesive system.

I acknowledge the open-source community for the technologies that made this project possible: Flutter, FastAPI, OpenStreetMap, OSRM, SQLite, and MBTiles. These tools embody principles of openness and accessibility that align with the goals of rural tourism development.

Finally, I dedicate this work to the communities and destinations of Gandaki Province, Nepal, whose tourism potential I hope this application helps to advance.

Contents

1	Introduction	1
1.1	Background and Motivation	1
1.2	Problem Statement	1
1.3	Project Aim and Objectives	2
1.4	Scope and Limitations	3
1.5	Report Structure	4
2	Literature Review	5
2.1	Rural Tourism in Nepal and Gandaki Province	5
2.2	Offline-First Mobile Application Architecture	5
2.3	Recommendation Systems in Tourism	6
2.4	Natural Language Processing and Chatbots for Tourism	7
2.5	Related Work in Tourism Applications	8
3	System Design and Architecture	10
3.1	System Architecture Overview	10
3.2	Feature-Based Code Organisation	12
3.3	Offline-First Routing Architecture	13
3.4	Data Architecture and Storage	15
3.4.1	Local Asset Data	15
3.4.2	Destination Images	16
3.4.3	SQLite and Hive Persistence	16
3.4.4	SharedPreferences	17
3.4.5	Backend API Data	17
3.5	State Management with Riverpod	17
4	Implementation	19
4.1	Technology Stack	19
4.2	Offline Map Implementation	20
4.2.1	Map Tile Generation	20
4.2.2	Map Validation and Loading	20

4.2.3	Map Controls and Styling	21
4.3	Routing Service Implementation	22
4.4	Chatbot Implementation	23
4.5	Recommendation Engine	27
4.6	Destination Data Curation	29
4.7	Key Application Screens	29
4.7.1	Home Screen	30
4.7.2	Destination Browsing	30
4.7.3	Destination Detail	30
4.7.4	Map Screen	30
4.7.5	Chatbot Screen	30
4.7.6	Recommendations Screen	30
4.7.7	Saved Places Screen	30
4.7.8	Account/Preferences Screen	31
4.7.9	Settings Screen	31
4.7.10	Onboarding Screen	31
4.7.11	About Screen	31
5	Evaluation	32
5.1	Evaluation Methodology	32
5.2	Offline Functionality Testing	33
5.3	Software Verification and Performance	34
5.3.1	Automated Verification Results	34
5.3.2	APK Size Analysis	34
5.4	Quantitative Evaluation Results	35
5.5	Objective Traceability and Evaluation Validity	36
5.6	Usability Evaluation	37
5.7	Comparison with Related Work	37
5.8	Limitations and Critical Analysis	38
5.8.1	Limited Destination Dataset	38
5.8.2	Limited Offline Map Zoom	39
5.8.3	Approximate Offline Routing	39
5.8.4	Limited Collaborative Data	39
5.8.5	Language Limitations	40
5.8.6	Device Storage Dependency	40
5.8.7	Lack of Real-Time Data	40
5.9	Societal Impact and Contribution	40
5.9.1	Information Accessibility	40
5.9.2	Economic Potential	40

5.9.3	Offline-First Design Pattern Contribution	41
5.9.4	Open-Source Potential	41
5.10	Ethical Considerations	41
5.10.1	Data Privacy	41
5.10.2	Accessibility	41
5.10.3	Cultural Sensitivity	41
5.10.4	Fair Attribution	41
6	Challenges and Solutions	42
6.1	Map Tile Format and Compatibility	42
6.2	MBTiles Coordinate System	42
6.3	APK Size Management	42
6.4	Offline Routing Fallback	43
6.5	Vector vs. Raster Tile Performance	43
7	Conclusion	44
7.1	Summary of Achievements	44
7.2	Contribution to Knowledge	45
7.2.1	Offline-First Architecture for Tourism	45
7.2.2	Rural Tourism Digitalisation	45
7.2.3	Mobile Application Design for Low-Connectivity Contexts	45
7.2.4	Open Dataset and Methodology	45
7.3	Future Work	45
7.3.1	Real User Feedback and Iteration	45
7.3.2	Expanded Destination Dataset	46
7.3.3	Full-Length Offline Routing	46
7.3.4	Expanded Multilingual Support	46
7.3.5	Community Contributions	46
7.3.6	iOS Support	46
7.3.7	Recommendation Learning from Real Users	46
7.3.8	Integration with Nepal Tourism Services	46
7.4	Reflection on Project Outcomes	47
7.5	Final Remarks	47
A	Sample Destination Data	50
B	API Endpoint Specifications	51
C	Testing Checklist	52
D	Database Schema Diagrams	53

E	System Requirements	54
F	Requirements Catalogue and Traceability	55
F.1	Offline Launch and Core Discovery	59
F.2	Bundled Destination Catalogue Integrity	60
F.3	Offline Map Availability	61
F.4	Three-Tier Route Resolution	62
F.5	Persistent Saved Destinations and Reviews	63
F.6	Offline Recommendation Continuity	64
F.7	Local Tourism Assistance and Safety	65
F.8	Backend Failure Degradation	66
G	Data Dictionary and Dataset Evidence	67
G.1	Dataset Identity and Uniqueness	69
G.2	Geographic Validity	70
G.3	Semantic Retrieval Readiness	71
G.4	Local Media Integrity	72
G.5	Accommodation Relationship Integrity	73
H	Detailed API Contract Catalogue	74
H.1	Health Check Contract	75
H.2	Destination Collection Contract	76
H.3	Destination Detail and Accommodation Contract	77
H.4	Recommendation Contract	78
H.5	Similar Destination Contract	79
H.6	Interaction Logging Contract	80
H.7	Chat Contract	81
H.8	Offline Chat Contract	82
H.9	Translation Contract	83
H.10	Authentication Contract	84
I	Verification Test Catalogue	85
I.1	Cold Offline Startup Test	86
I.2	Warm Online-to-Offline Transition Test	87
I.3	Offline-to-Online Recovery Test	88
I.4	Route Cache Precedence Test	89
I.5	Haversine Fallback Test	90
I.6	Recommendation Scenario Evaluation Test	91
I.7	Recommendation Explanation Test	92
I.8	Cold-Start Recommendation Test	93

I.9	Emergency Detection Test	94
I.10	Dialogue Continuity Test	95
I.11	Translation Coverage Test	96
I.12	Static Analysis and Automated Regression Test	97
J	User Manual and Operational Workflows	98
J.1	First Launch and Onboarding	99
J.2	Browse and Search Destinations	100
J.3	Read Destination Details	101
J.4	Save and Review a Destination	102
J.5	Explore the Offline Map	103
J.6	Request Directions	104
J.7	Receive Personalized Recommendations	105
J.8	Use the Tourism Chatbot	106
J.9	Translate a Travel Phrase	107
J.10	Recognise Offline and Recovery States	108
K	Deployment, Configuration, and Troubleshooting	109
K.1	Flutter Development Setup	110
K.2	Backend Development Setup	111
K.3	Backend URL and Environment Switching	112
K.4	Android Release Build and Asset Verification	113
K.5	Operational Troubleshooting Guide	114
L	Architecture Decision Records	115
L.1	Use Flutter for the Mobile Client	116
L.2	Treat Offline Operation as the Default Architecture	117
L.3	Use FastAPI for Online Enhancement Services	118
L.4	Combine SQLite, Hive, and SharedPreferences	119
L.5	Bundle MBTiles for Offline Map Coverage	120
L.6	Use Cache, OSRM, then Haversine for Routing	121
L.7	Hybrid Explainable Recommendation Pipeline	122
L.8	Keep Deterministic Local Safety and Intelligence	123
M	Selected Implementation Listings	124
M.1	FastAPI Recommendation Route	125
M.2	Backend Recommendation Application Service	128
M.3	Backend Contextual Reranker Excerpt	133
M.4	Flutter Routing Service	138
M.5	Offline Semantic Encoder	142

M.6 Intelligence Orchestrator Excerpt	147
M.7 Backend Collaborative Filtering Test	153
M.8 Flutter Intelligence End-to-End Test	154
N Project Governance, Risk, and Evidence Index	155
N.1 Ethical and Responsible-Use Checklist	157
N.2 Project Milestone Record	158
N.3 Submission Evidence Index	159

List of Figures

3.1	Layered system architecture. The application separates presentation logic, domain models, data persistence, and core services. Services access both local resources (storage, assets) and optional external backend services. This separation enables offline-first operation.	10
3.2	Feature-based code organisation. Each feature module contains related screens, models, and widgets. The core/services layer provides shared functionality to all features.	13
3.3	Three-tier routing architecture. The system checks the SQLite route cache first, then attempts online OSRM routing if needed. If OSRM fails, a Haversine straight-line fallback is computed. All paths guarantee a route result to the user.	15
5.1	Precision, recall, and ranking quality across five tourism scenarios.	35
D.1	Simplified SQLite schema for Paila Nepal. The main application database stores saved destinations and reviews, while a separate SQLite database stores the 30-day route cache. Recommendation caches and pending interactions use Hive.	53

List of Tables

2.1	Comparison of tourism applications and Paila Nepal. “Offline Maps” indicates whether offline map display is supported. “Gandaki Focus” indicates rural Gandaki destination specialisation. “Chatbot” indicates conversational tourism assistance. “Recommen.” indicates recommendation system. “Routing” indicates route guidance. “Saved” indicates saved place functionality.	9
4.1	Primary technology stack for Paila Nepal. The application uses Flutter/Dart for mobile development, SQLite and Hive for local persistence, and FastAPI for backend services.	19
4.2	Sample destination records from the Gandaki dataset. Each destination includes 15+ attributes for filtering and recommendation.	30
5.1	Development-stage offline functionality checks. OSRM and online LLM enhancement are intentionally unavailable offline, while local fallbacks preserve core functionality.	33
5.2	Repeatable automated verification results from the final report pass.	34
5.3	Recommendation-quality results across the five dissertation scenarios.	35
5.4	System Usability Scale (SUS) items used for usability evaluation.	38
5.5	Feature comparison of tourism applications. Legend: ●● = full support, ● = partial support, Not supported = not supported. Paila Nepal uniquely combines offline-first architecture with rural Gandaki tourism specialisation.	39
A.1	Sample destination data structure showing attributes included for each of 300 Gandaki Province destinations.	50
F.1	Functional requirements catalogue.	55
F.2	Non-functional requirements catalogue.	57
G.1	Destination data dictionary.	67
N.1	Project risk register.	155

Abbreviations

Abbreviation	Meaning
API	Application Programming Interface
APK	Android Package
CRUD	Create, Read, Update, Delete
GPS	Global Positioning System
HTTP	Hypertext Transfer Protocol
IDE	Integrated Development Environment
JSON	JavaScript Object Notation
MBTiles	Map Box Tiles
FAISS	Facebook AI Similarity Search
Hive	Local key-value database used by the Flutter application
MRR	Mean Reciprocal Rank
nDCG	Normalised Discounted Cumulative Gain
NLP	Natural Language Processing
OSRM	Open Source Routing Machine
RAG	Retrieval-Augmented Generation
SBERT	Sentence-BERT
SQLite	Structured Query Language Lite
UI	User Interface
UX	User Experience
WebP	Efficient modern image format
XYZ	Tile coordinate system (tile.openstreetmap.org/z/x/y)
TMS	Tile Map Service

Chapter 1

Introduction

1.1 Background and Motivation

Tourism is an important economic and cultural sector in Nepal, generating employment, foreign exchange, and opportunities for regional development (Nepal Tourism Board, [2023](#)). Gandaki Province contains Pokhara, a major tourism gateway, alongside rural villages, mountains, lakes, trekking routes, cultural settlements, and pilgrimage sites. Yet digital coverage is concentrated on Pokhara and famous routes, while many rural destinations remain poorly represented.

This information gap is intensified by unreliable connectivity. Travelers moving through rural Gandaki frequently encounter weak mobile data, high bandwidth costs, power disruption, and inconsistent internet access at accommodation. Most mainstream tourism platforms assume a continuous connection: they download content on demand, stream images, and request live routes. When the network fails, maps, descriptions, accommodation details, recommendations, and assistance can disappear together.

Paila Nepal is motivated by the principle that rural travel support should remain useful without continuous internet access. It therefore treats locally stored information and computation as the operational foundation, using the network to enhance results when available. This is the central idea of offline-first design.

1.2 Problem Statement

Rural travelers in Gandaki Province face three connected problems. First, geographic context and navigation can disappear when map tiles and live route requests fail. Second, many destinations lack structured information about activities, accessibility, budgets, seasons, culture, family suitability, adventure level, and nearby accommodation. Third, general-purpose maps, booking platforms, and review aggregators do not prioritise Gandaki's villages, homestays, viewpoints, or low-connectivity travel conditions. These weak-

nesses create uncertainty for travelers and reduce the visibility and potential economic benefit of rural destinations.

The core problem addressed by this project can be stated as follows:

How can a mobile application provide offline-capable destination discovery, map access, route guidance, personalized recommendations, and tourism assistance for Gandaki Province rural tourism, particularly in contexts where internet connectivity is unreliable or unavailable?

Addressing the problem requires more than cached maps. It requires bundled destination data and media, local map tiles, route caching and fallback computation, local tourism intelligence, persistent storage, meaningful destination attributes, and optional online enhancement.

1.3 Project Aim and Objectives

The primary aim of this project is to design and implement a Flutter-based Android mobile application that provides offline-first rural tourism support for Gandaki Province, demonstrating how offline-first architecture can enhance digital accessibility in low-connectivity tourism contexts.

The specific objectives are:

1. To build a fully offline-capable Flutter mobile application for Android that launches, displays destination data, supports map exploration, provides route guidance, and answers tourism queries without requiring internet connectivity.
2. To curate and integrate a structured dataset of 300 rural tourism destinations in Gandaki Province, including geographic coordinates, categories, activities, seasonality, budget levels, accessibility ratings, family-friendliness indicators, adventure levels, cultural attributes, nature attributes, descriptions, tags, and image references.
3. To bundle local destination photographs using optimised WebP compression (approximately 300 images), allowing destination cards, list views, and detail screens to display rich visual content offline.
4. To integrate a local MBTiles raster map package focused on Gandaki Province (zoom levels 7–12), enabling users to browse Pokhara and surrounding rural areas without downloading tiles at runtime.
5. To implement a three-tier routing architecture: (1) SQLite route cache for previously-requested routes, (2) online OSRM routing when internet is available, and (3) offline Haversine straight-line route computation as a fallback.

6. To implement an offline-capable chatbot using local language processing, hybrid intent classification, knowledge retrieval, dialogue memory, emergency detection, structured response generation, and optional online LLM enhancement.
7. To provide explainable personalized destination recommendations using semantic embeddings, contextual matching, cold-start handling, diversity filtering, interaction-derived collaborative signals, and machine-learning reranking.
8. To support persistent user data including saved destinations, reviews, recommendation caches, queued interactions, and preferences through SQLite, Hive, and Shared-Preferences.
9. To integrate a FastAPI backend for online enhancements including AI chatbot responses, advanced recommendations, user authentication, and analytics, while preserving core local functionality.
10. To evaluate the application through offline functionality testing, performance analysis, user experience assessment, and critical evaluation of design decisions and limitations.

1.4 Scope and Limitations

The project focuses on Gandaki Province, particularly destinations accessible from Pokhara. Its 300 curated destination records provide meaningful coverage while remaining practical for offline distribution. The bundled raster map covers zoom levels 7–12; extending this to street-level detail across every village would make the application substantially larger.

Routing uses OSRM for road-aware instructions when online, cached routes when available, and a Haversine approximation when disconnected. The fallback is useful for direction and distance estimation, but it is not offline turn-by-turn navigation. The local chatbot does not run an LLM on-device; instead, it combines multilingual text processing, hybrid intent classification, retrieval, dialogue state, safety rules, and templates, with optional cloud-model enhancement.

The deployment target is Android. iOS testing and distribution are outside the present scope. Other limitations include best-available photographs that may be representative rather than exact, no real-time traffic or weather, incomplete community-contribution workflows, and no bundled road graph for fully offline navigation.

1.5 Report Structure

The remaining chapters review relevant literature, present the architecture and implementation, evaluate the completed system, discuss technical challenges, and conclude with contributions, limitations, and future work.

Chapter 2

Literature Review

2.1 Rural Tourism in Nepal and Gandaki Province

Rural tourism links Nepal’s landscapes and cultural heritage with community-based economic development. Unlike tourism concentrated in cities or famous routes, it can distribute spending to homestays, local guides, small guesthouses, craft producers, and agricultural communities. Gandaki Province is especially important because Pokhara provides a gateway to villages such as Ghandruk, Ghorepani, Bandipur, and Sikles, as well as trekking, lake, adventure, pilgrimage, and cultural experiences.

Tourism can support livelihoods, infrastructure, conservation, and cultural preservation, but it can also create environmental pressure, dependency, and cultural commodification. Accessible and accurate information is therefore important: travelers need to understand routes, accommodation, seasons, activities, costs, culture, and safety before choosing lesser-known places. Many rural Gandaki destinations remain poorly described online, and their information often becomes inaccessible when connectivity fails.

Paila Nepal treats rural destination information as core local content. Structured attributes, photographs, maps, and supporting knowledge remain available offline and can also support recommendation and conversational assistance.

2.2 Offline-First Mobile Application Architecture

Offline-first architecture treats local data and computation as the operational foundation rather than treating disconnection as an exceptional error. The network becomes a channel for synchronisation, remote computation, fresh data, and improved results. Common patterns include reading local data first, applying local updates immediately, queueing remote operations, resolving conflicts after reconnection, communicating synchronisation state, and degrading unavailable services to useful local alternatives. SQLite suits durable relational records because it is embedded and serverless (SQLite, 2026), while Flutter pro-

vides one mobile codebase with access to storage and mapping packages (Flutter, 2026).

Paila Nepal applies these patterns by bundling 300 destination records and local WebP images, storing raster maps in MBTiles, caching routes in SQLite, computing a Haversine fallback, running local tourism intelligence, and persisting saved places, reviews, preferences, and queued interactions. This combination allows the user interface and core local workflows to remain available while remote services are unreachable.

MBTiles is central to the mapping strategy because it stores tiled map data in one SQLite-based container rather than thousands of separate files. This simplifies distribution, validation, and version management. The map derives from OpenStreetMap data (OpenStreetMap Foundation, 2026), while OSRM supplies road-network routes when online (Open Source Routing Machine Project, 2026). The project’s `pokhara.mbtiles` file is approximately 7.79 MB and covers the focused area at several zoom levels.

2.3 Recommendation Systems in Tourism

Recommendation systems help users navigate large collections by suggesting items predicted to be relevant or interesting. In tourism contexts, recommender systems can suggest destinations, attractions, accommodations, activities, routes, or trip plans based on user preferences and contextual factors.

Tourism recommendation is more complex than recommending generic products because travel decisions involve multiple interconnected dimensions: geographic distance, seasonality, budget, activity preferences, cultural interests, family composition, physical accessibility, adventure tolerance, weather conditions, and personal mood. A destination that is ideal for a solo trekker may be unsuitable for a family with young children. A destination attractive in autumn may be dangerous during monsoon. Budget-conscious tourists need different recommendations than luxury travelers.

The first prototype of Paila Nepal used content-based filtering, in which destination attributes were compared directly with user preferences. This was appropriate during early development because content-based methods remain usable during cold start, when a newly installed application has no interaction history. However, a purely content-based design risks overspecialisation and cannot learn effectively from user behaviour. Recommender-system literature therefore identifies hybridisation as a practical way to combine the strengths of different signals while reducing their individual weaknesses (Adomavicius and Tuzhilin, 2005; Burke, 2002).

The destination dataset supports content-based filtering through category, activities, best season, budget, accessibility, adventure level, family suitability, cultural richness, nature score, tags, and descriptive text.

A recommendation algorithm computes a match score between user preferences and each destination’s attributes. For example, if a user indicates preferences for “trekking

in autumn with moderate budget and family-friendly options,” the system can score destinations by measuring alignment with these criteria. High-scoring destinations are recommended first. Diversity constraints can ensure that recommendations include varied destination types rather than clustering on a single category.

The final Paila Nepal recommender is hybrid. On the backend, a natural-language preference query is encoded using Sentence-BERT, which is designed to produce semantically meaningful sentence embeddings suitable for similarity comparison (Reimers and Gurevych, 2019). Candidate destinations are retrieved through a normalised embedding index using FAISS when available and a NumPy dot-product fallback otherwise. Retrieval scores are then combined with structured activity and category signals. A collaborative component uses stored interactions when a user has sufficient history; popularity provides a limited cold-start prior. A contextual reranker evaluates season, budget, activity, vibe, accessibility, family suitability, adventure preference, and accommodation fit. Finally, a trained LightGBM LambdaRank model can rescore the candidates, while a fixed-weight fallback preserves operation when no trained model is available. The application also generates human-readable reasons so that users can inspect why a destination was recommended.

The mobile offline recommender uses precomputed destination embeddings and contextual attributes stored in application assets. Its first stage combines semantic embedding similarity with contextual fit, after which constraint penalties, local affinity signals, and diversity limits are applied. This differs from the earlier TF-IDF and simple cosine-similarity prototypes discussed in the interim submission: those prototypes established the feasibility of local ranking, while the final implementation uses richer semantic vectors and a broader set of contextual signals. TF-IDF nevertheless remains an important baseline because it represents text through weighted term frequency rather than semantic meaning (Salton and Buckley, 1988).

Tourism recommendation literature emphasises the importance of context and personalisation. Tkalčič and Chen, 2019 discuss context-aware recommendation systems, emphasising how travel recommendations must account for season, weather, company (solo, family, group), and activity preferences. Ye, Yin, and Lee, 2010 examine location-based recommendation systems, noting that geographic proximity and user activity patterns significantly influence tourism recommendations.

2.4 Natural Language Processing and Chatbots for Tourism

Chatbots in tourism applications serve various functions: answering frequently-asked questions, providing contextual information, facilitating bookings, offering safety guid-

ance, and providing translation services. Advanced chatbots using large language models (LLMs) can engage in fluent natural conversation and provide nuanced responses to varied questions.

However, cloud-based LLMs create a dependency on internet connectivity and API availability. In a system designed for offline-first rural tourism, relying entirely on cloud AI would reproduce the same connectivity vulnerability that the project is addressing. A practical alternative is to implement local natural language processing using bounded resources.

Bounded local chatbots generally combine a domain knowledge base, intent classification, entity extraction, response templates, and fallback handling. Their scope is narrower than a cloud LLM, but their behaviour is predictable, testable, private, and independent of API availability.

The earliest chatbot prototype used a local knowledge base, keyword matching, and structured templates. The final application expands this into a modular offline intelligence pipeline. English, Devanagari Nepali, and romanised Nepali text is normalised, tokenised, stemmed, enriched with synonyms, and analysed for named entities. Hybrid intent classification and semantic/BM25-style retrieval produce grounded answers, while dialogue state, slot filling, clarification questions, and memory support multi-turn interaction. It answers questions about seasons, budgets, family suitability, transport, and activities. A safety layer runs before normal response generation, ensuring that messages such as “I’m lost” or “I’m injured” receive deterministic emergency guidance.

When backend connectivity and API keys are configured, the chatbot can request online language-model enhancement. The enhancement is applied to the locally generated answer rather than replacing the complete local pipeline. However, the offline chatbot remains the reliable base layer. This design follows the offline-first principle: cloud enhancement is available when possible, but is not required for core operation. The implementation also distinguishes backend reachability from provider availability so that the interface does not incorrectly imply that online AI is active when only fallback logic is available.

2.5 Related Work in Tourism Applications

Several existing applications and systems are relevant to Paila Nepal:

Google Maps provides extensive navigation and points of interest, but its richer services remain connectivity-dependent and are not focused on rural destination discovery. TripAdvisor supplies reviews and commercial tourism content but is primarily online. OSMAnd provides strong offline maps and navigation without curated Gandaki tourism assistance. Komoot and AllTrails support outdoor route discovery, yet focus mainly on hiking and community trails. Local tourism-board guides may provide regional informa-

tion, but their data coverage and offline functionality vary.

table 2.1 compares these systems against Paila Nepal along several dimensions.

System	Offline Maps	Gandaki Focus	Chatbot	Recommen.	Routing	Saved
Google Maps	Partial	No	No	No	Yes	Yes
TripAdvisor	No	No	No	Yes	No	Yes
OSMAnd	Yes	No	No	No	Yes	No
Komoot	Yes	No	No	No	Yes	Yes
AllTrails	Partial	No	No	No	No	Yes
Paila Nepal	Yes	Yes	Yes	Yes	Yes	Yes

Table 2.1: Comparison of tourism applications and Paila Nepal. “Offline Maps” indicates whether offline map display is supported. “Gandaki Focus” indicates rural Gandaki destination specialisation. “Chatbot” indicates conversational tourism assistance. “Recommen.” indicates recommendation system. “Routing” indicates route guidance. “Saved” indicates saved place functionality.

The distinctive contribution of Paila Nepal is the integration of offline-first architecture, rural Gandaki tourism content specialisation, and comprehensive offline functionality (maps, routing fallback, chatbot, recommendations). This combination addresses the specific needs of rural tourism in low-connectivity environments, a gap not fully addressed by existing applications.

Chapter 3

System Design and Architecture

3.1 System Architecture Overview

Paila Nepal follows a layered architecture separating presentation, domain, data, and service layers. This separation of concerns improves testability, maintainability, and enables offline/online behavior to be managed cleanly. fig. 3.1 illustrates the layered architecture.

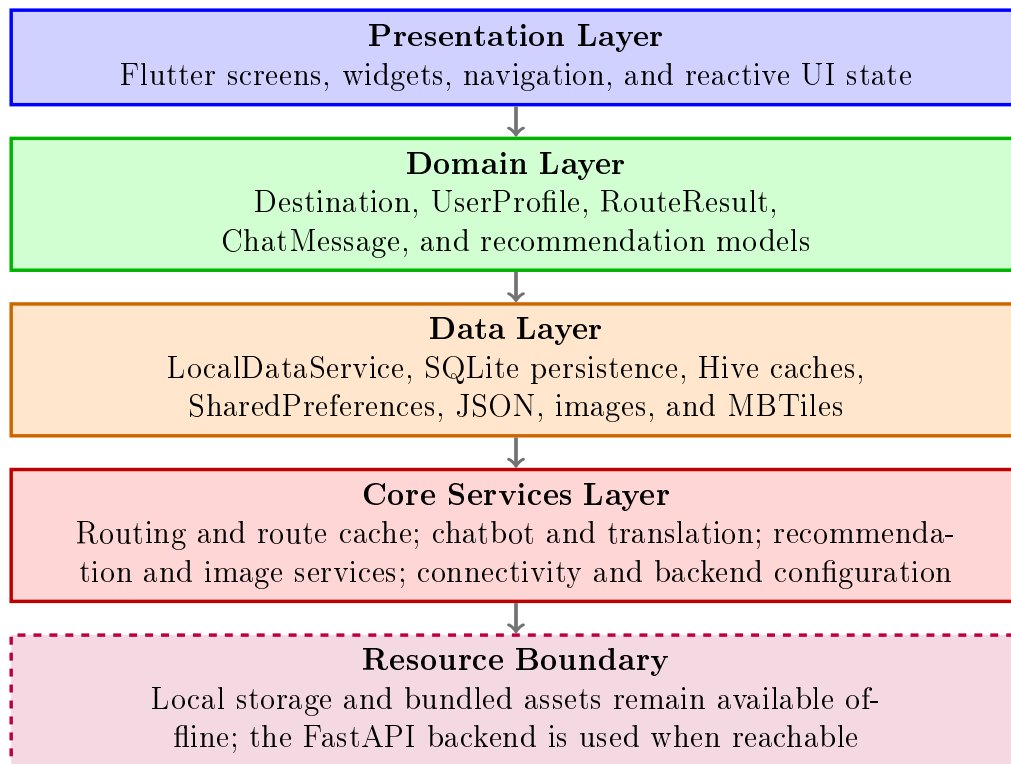


Figure 3.1: Layered system architecture. The application separates presentation logic, domain models, data persistence, and core services. Services access both local resources (storage, assets) and optional external backend services. This separation enables offline-first operation.

The **Presentation Layer** consists of Flutter widgets, screens, UI components, dialogs, bottom sheets, and navigation logic. Examples include the home screen, destination

list, destination detail view, map screen, chatbot interface, recommendations screen, and settings screens. The presentation layer receives input from users and displays output by rendering widgets. It communicates with services and providers to retrieve data and trigger actions.

The **Domain Layer** contains entities and models representing core application concepts. Examples include:

- **Destination** model with attributes: `id`, `name`, `description`, `latitude`, `longitude`, `category`, `activities`, `best_season`, `budget_level`, `accessibility`, `adventure_level`, `family_friendly`, `cultural_richness`, `nature_score`, `tags`, `image_url`.
- **UserProfile** model with preferences: `interests`, `budget`, `adventure_level`, `family_status`.
- **RouteResult** model with: `origin`, `destination`, `distance`, `duration`, `polyline`, `steps`, `is_fallback`.
- **ChatMessage** model with: `content`, `sender` (user/chatbot), `timestamp`, `intent`.
- **RecommendationResult** model with: `recommended_destinations`, `scores`, `reasoning`.

Domain models define the structure of data independently of UI or persistence concerns. This ensures that data can move between services, screens, and storage mechanisms in a type-safe manner.

The **Data Layer** manages all local and remote data access. It includes:

- **LocalDataService**: Loads destinations and accommodations, manages SQLite records for saved destinations, reviews, events, and profiles, and coordinates Hive-backed recommendation data.
- **Asset loading**: JSON files containing destination data, chatbot knowledge bases, phrasebooks, embeddings.
- **HiveStorageService**: Stores recommendation caches, user preferences, local interactions, and interactions queued for backend synchronisation.
- **SharedPreferences**: Lightweight key-value storage for settings and web fallbacks.
- **Image services**: Access to destination images stored as WebP assets and cached network images.
- **MBTiles provider**: Access to offline map tiles via `flutter_map_mbtiles`.

The **Services Layer** contains cross-cutting logic that coordinates lower layers:

- **RoutingService:** Coordinates route retrieval from cache, online OSRM, or fallback Haversine computation.
- **IntelligenceOrchestrator:** Coordinates multilingual normalisation, intent classification, entity recognition, hybrid retrieval, dialogue state, safety checks, translation, and optional online enhancement.
- **OfflineTileProvider:** Validates and loads MBTiles file for map display.
- **ImageService:** Coordinates image loading from assets, cache, or backend.
- **OfflineProvider** and **BackendConfig:** Distinguish general connectivity, backend reachability, and the availability of online enhancement.
- **RecommenderService** and **RecommendationApiService:** Provide offline semantic/-contextual ranking and access to the backend hybrid recommender.
- **InteractionSyncService:** Queues local interactions and synchronises them in batches when the backend becomes available.
- **BackendConfigService:** Manages backend API configuration, authentication, and availability.

This layered architecture supports offline-first operation because most presentation features depend on local services and persistent data first. The backend provides enhancement and synchronisation but is not a dependency for core functionality.

3.2 Feature-Based Code Organisation

In addition to the layered architecture, Paila Nepal uses feature-based code organisation, grouping related screens, models, services, and widgets into feature modules. This improves modularity and allows individual features to be developed and tested semi-independently.

Key feature modules include:

- **features/destinations:** Destination browsing, search, filtering, detail view, gallery view, and saved destination management.
- **features/map:** Interactive map with route display, marker handling, map tile switching, offline map status, and navigation controls.
- **features/chatbot** and **features/intelligence:** Chatbot UI, offline NLP, hybrid intent classification, RAG-style knowledge retrieval, safety handling, dialogue state, translation, and optional online enhancement.

- **features/recommendations:** Personalised recommendation display, semantic/-contextual offline ranking, backend hybrid ranking, preference management, score explanations, and recommendation caching.
- **core/navigation and features/map:** Route request, route display, navigation UI, route caching, OSRM access, and offline route fallback indication.
- **features/auth:** User authentication UI, login/signup screens, session management, and backend integration.
- **features/account:** User profile, preferences, saved destinations, trip history, and settings.
- **features/home:** Home screen, discovery feed, quick access widgets, and onboarding.
- **features/shell:** Bottom navigation, tab layout, and overall app structure.

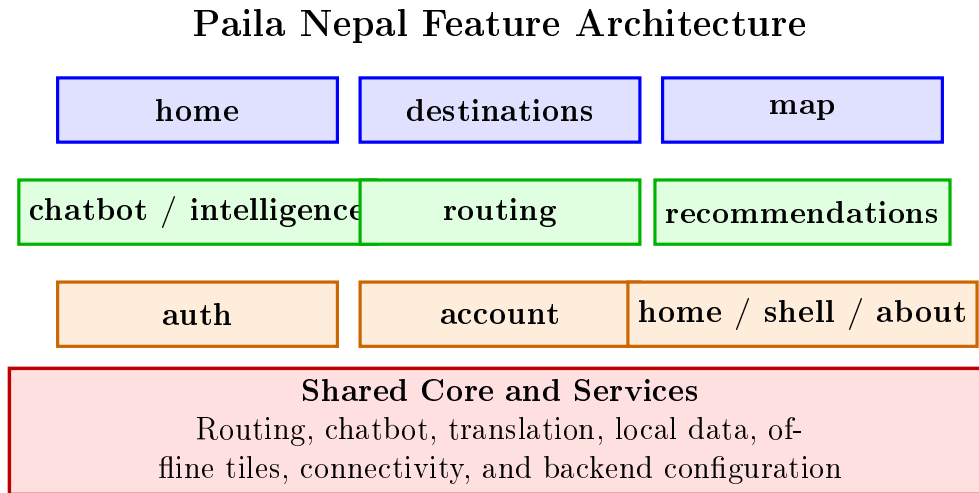


Figure 3.2: Feature-based code organisation. Each feature module contains related screens, models, and widgets. The core/services layer provides shared functionality to all features.

3.3 Offline-First Routing Architecture

The routing architecture is central to offline-first operation. It implements a three-tier strategy that ensures route guidance is always available, even when the online routing service is unreachable.

Tier 1: SQLite Route Cache. The application maintains a ‘RouteCache’ SQLite table storing previously-computed routes. The cache key deterministically encodes the origin coordinates, destination coordinates, and travel mode. When a user requests a

route, the system first checks the cache. If a valid cached entry exists (with a timestamp less than 30 days old), the cached route is returned immediately. This eliminates the need for a network request and provides instant route results.

Tier 2: Online OSRM Routing. If no valid cached route exists, the application attempts to retrieve an online route from the OSRM (Open Source Routing Machine) public demo server. The request includes:

- Origin and destination coordinates.
- `geometries=geojson` for polygon response format.
- `steps=true` for turn-by-turn instructions.
- Travel mode (driving, walking, cycling).
- HTTP timeout of 12 seconds.

If the OSRM request succeeds, the response is parsed to extract the route geometry (polyline), total distance, total duration, and step-by-step instructions. The route is then stored in the SQLite cache and returned to the user. The cache entry will provide instant results for subsequent requests to the same origin-destination pair, even if the device goes offline.

Tier 3: Haversine Straight-Line Fallback. If OSRM is unreachable (due to network failure, server timeout, or parsing error), the application falls back to an approximate offline route. It uses the Haversine formula to compute the great-circle distance between origin and destination. The displayed route polyline contains two points: origin and destination. Duration is estimated using travel-mode speed assumptions:

- Driving: 60 km/h.
- Walking: 5 km/h.
- Cycling: 15 km/h.

The estimated duration is computed as: $\text{duration} = \frac{\text{haversine_distance}}{\text{speed}}$.

The route result is marked with `'isFallback = true'`, allowing the UI to display a warning message such as: “This is an approximate route. Turn-by-turn navigation requires internet connectivity.” This manages user expectations clearly.

This architecture ensures that users always receive route guidance, even in severely constrained connectivity situations. Cached routes provide instant results for frequently-requested routes. OSRM provides high-quality road-network routes when connectivity allows. The Haversine fallback ensures the app never fails with an error, but instead provides approximate guidance with transparent warnings.

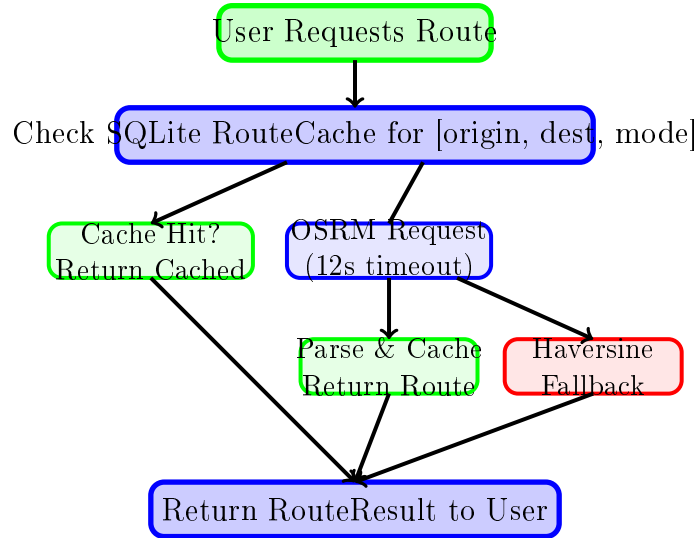


Figure 3.3: Three-tier routing architecture. The system checks the SQLite route cache first, then attempts online OSRM routing if needed. If OSRM fails, a Haversine straight-line fallback is computed. All paths guarantee a route result to the user.

3.4 Data Architecture and Storage

Paila Nepal uses multiple storage layers to support offline operation:

3.4.1 Local Asset Data

`assets/data/destinations.json` contains 300 curated destination records. Each destination includes:

```

{
  "id": "destination_001",
  "name": "Ghandruk",
  "description": "A scenic Gurung village in the Annapurna foothills...",
  "latitude": 28.3956,
  "longitude": 83.8622,
  "category": "village",
  "activities": ["trekking", "village_tour", "culture"],
  "best_season": ["autumn", "spring"],
  "budget_level": "budget",
  "accessibility": "moderate",
  "adventure_level": 2,
  "family_friendly": true,
  "cultural_richness": 8,
  "nature_score": 7,
  "tags": ["gandaki", "annapurna", "gurung"],
  "image_url": "ghandruk.webp"
}

```

```
}

```

This JSON is loaded into memory on app startup and remains accessible offline. The data structure supports efficient searching, filtering, and recommendation scoring.

3.4.2 Destination Images

`assets/destination_images/` contains 300 WebP format images (approximately 8.74 MB total). WebP provides better compression than JPEG or PNG, reducing the bundle size while maintaining visual quality suitable for mobile displays.

3.4.3 SQLite and Hive Persistence

The application uses two SQLite databases together with several Hive boxes. This separation reflects the different access patterns of each data type: relational records and indexed queries remain in SQLite, while typed caches and synchronisation queues use Hive.

Main App Database (`paila.db`):

Table	Columns	Purpose
<code>saved_destinations</code>	<code>id</code> , <code>payload</code> , <code>saved_at</code>	User-saved destinations
<code>app_events</code>	<code>id</code> , <code>event_type</code> , <code>payload</code> , <code>created_at</code>	Local application event log
<code>destination_reviews</code>	<code>destination_id</code> , <code>rating</code> , <code>review_text</code> , <code>created_at</code>	Local destination ratings and reviews
<code>user_profiles/interaction_profile</code>	<code>profile</code> and <code>interaction</code> fields	Personalisation history and migration support

Route Cache Database (`route_cache.db`):

Table	Columns	Purpose
<code>cached_routes</code>	<code>key</code> , <code>data</code> , <code>cached_at</code>	Route caching (30-day TTL)

Hive Boxes:

- `recommendation_cache_v1`: cached recommendation payloads and timestamps.
- `user_preferences_v1`: the current structured recommendation preferences.
- `user_interactions_v1`: local interaction history used for personalisation.
- `pending_backend_interactions_v1`: queued events waiting for batch synchronisation.

3.4.4 SharedPreferences

Lightweight key-value storage for:

- Selected map style.
- Lightweight settings and web-platform fallbacks.
- Last app version opened.
- Offline map validation status.
- User authentication token (if available).

3.4.5 Backend API Data

When connectivity is available, the FastAPI backend can provide:

- Enhanced chatbot responses via LLM integration.
- Hybrid recommendations using SBERT retrieval, collaborative and popularity signals, contextual reranking, and optional ML rescoring.
- User authentication and account synchronisation.
- Interaction batch synchronisation, analytics, recommendation-quality monitoring, and model training/evaluation endpoints.
- Image lookup and CDN delivery.
- Trip planning suggestions.

The backend is optional for core local workflows. It enhances the experience with road-aware routing, online AI, account services, synchronisation, analytics, and the most advanced recommendation pipeline when connectivity is available.

3.5 State Management with Riverpod

Paila Nepal uses Flutter Riverpod for reactive state management and dependency injection. Riverpod provides a provider-based architecture in which widgets observe state changes without tight coupling to service implementations.

Example providers:

```
// Async data provider
final destinationsProvider = FutureProvider<List<Destination>>((ref) async {
    final service = ref.watch(localDataServiceProvider);
    return service.loadDestinations();
});

// Filtered destinations based on criteria
final filteredDestinationsProvider =
    FutureProvider.family<List<Destination>, FilterCriteria>((ref, criteria) async {
        {
            final destinations = await ref.watch(destinationsProvider.future);
            return destinations.where((d) => d.matchesCriteria(criteria)).toList();
        }
    });

// Recommendations
final recommendationsProvider =
    FutureProvider<List<Destination>>((ref) async {
        final prefs = ref.watch(userPreferencesProvider);
        final engine = ref.watch(recommendationEngineProvider);
        return engine.generateRecommendations(prefs);
    });

// Async route computation
final routeProvider = FutureProvider.family<RouteResult, RouteRequest>((ref,
    request) async {
    final routing = ref.watch(routingServiceProvider);
    return routing.getRoute(request);
});
```

Local widget state (`StatefulWidget`) is used for transient UI state such as animation controllers, selected filters, text input, and map controller interactions. This mixed approach balances reactivity with simplicity.

Chapter 4

Implementation

4.1 Technology Stack

table 4.1 summarises the primary technologies used in Paila Nepal.

Component	Technology	Version/Details
Framework	Flutter	SDK $\geq$ 3.0.0
Language	Dart	3.x
State Management	flutter_riverpod	3.3.1
Maps	flutter_map	8.2.2
Offline Tiles	flutter_map_mbtils	1.0.4
Database	sqflite	2.3.3+1
Typed Local Cache	Hive / hive_flutter	2.2.3 / 1.1.0
Connectivity	connectivity_plus	7.1.1
Location Services	geolocator	14.0.1
HTTP Networking	http	1.2.0
Image Caching	cached_network_image	3.4.1
UI Fonts	google_fonts	6.2.1
Analytics	PostHog + Sentry	posthog_flutter 5.24.2, sentry_flutter 9.0.0
Speech	flutter_tts + speech_to_text	3.8.5 / 7.0.0
Backend	FastAPI	Python 3.9+
Backend Database	PostgreSQL	with SQLite fallback
Semantic Retrieval	Sentence-BERT + FAISS	NumPy fallback when FAISS is unavailable
Learning-to-Rank	LightGBM LambdaRank	Fixed weighted fallback before training
Backend Cache	Redis	In-memory fallback for local operation

Table 4.1: Primary technology stack for Paila Nepal. The application uses Flutter/Dart for mobile development, SQLite and Hive for local persistence, and FastAPI for backend services.

Flutter was selected as the primary framework because it enables rapid development of polished mobile UIs, provides consistent rendering across devices, supports hot reload for development iteration, and offers plugin support for databases, location, connectivity, and maps (Flutter, 2026).

Dart provides null-safety, `async/await` syntax, and strong type checking, making it well-suited for building reliable mobile applications.

Riverpod provides reactive state management, enabling UI to react to data changes without tightly coupling presentation logic to service implementations.

flutter_map provides open-source map rendering using OpenStreetMap data, without depending on proprietary Google Maps SDK or requiring API keys.

flutter_map_mbtils enables offline map display by reading raster tiles from local MBTiles files.

sqflite provides SQLite support for relational local records, while **Hive** provides typed boxes for recommendation caches, preferences, local interactions, and queued synchronisation events.

connectivity_plus detects network availability, allowing the app to determine whether it should use online or offline-only paths.

FastAPI on the backend provides a typed Python framework for API endpoints, automatic OpenAPI documentation, and straightforward integration with the mobile client (FastAPI, 2026).

4.2 Offline Map Implementation

The offline map implementation uses a local MBTiles file (`pokhara.mbtiles`, approximately 7.79 MB) containing raster PNG tiles for the focused project area. The MBTiles format is a SQLite database containing map tiles, metadata, and other data.

4.2.1 Map Tile Generation

Map tiles were generated by downloading raster PNG tiles from the OpenStreetMap tile server for the Gandaki bounding box (approximately 83.4°E to 85.2°E, 27.8°N to 29.2°N). Tiles at zoom levels 7–12 were selected to provide regional overview to destination-level detail. The tiles were aggregated into a single MBTiles SQLite database.

The MBTiles file structure includes:

- **Metadata table:** Key-value pairs storing map name, type, version, attribution, and bounds.
- **Tiles table:** Records containing zoom level (z), tile column (x), tile row (y), and tile data (PNG bytes).

4.2.2 Map Validation and Loading

Before using the MBTiles file, the app validates it:

```
Future<bool> validateMBTiles(String filePath) async {
  try {
    final file = File(filePath);
    if (!file.existsSync()) return false;

    final stat = file.statSync();
    if (stat.sizeSync() < 1000000) return false; // < 1MB is suspicious

    // Check SQLite magic header
    final bytes = file.readAsBytesSync().take(16).toList();
    final signature = String.fromCharCode(bytes.take(13));
    return signature.startsWith('SQLite format');
  } catch (e) {
    return false;
  }
}
```

If validation succeeds, the app loads tiles using the `flutter_map_mbtiles` provider. If validation fails or the file is missing, the app falls back to online tile providers (OpenStreetMap, CartoDB).

4.2.3 Map Controls and Styling

The map UI includes:

- Zoom controls (plus/minus buttons).
- Current location button (centers map on user's GPS position).
- Map style selector (enabling users to switch between Voyager, Positron, Dark Matter, or offline styles).
- Scale indicator.
- Offline/online status indicator showing whether local MBTiles is currently being used.
- Destination markers for browsable destinations.
- Route polyline display overlaid on the map.

4.3 Routing Service Implementation

The `RoutingService` orchestrates the three-tier routing strategy:

```
class RoutingService {
    final RouteCache routeCache;
    final http.Client httpClient;

    Future<RouteResult> getRoute(RouteRequest request) async {
        // Tier 1: Check cache
        final cacheKey = _generateCacheKey(request);
        final cached = await routeCache.get(cacheKey);
        if (cached != null && !cached.isExpired) {
            return cached;
        }

        // Tier 2: Try OSRM online
        try {
            final result = await _fetchOSRMRoute(request)
                .timeout(Duration(seconds: 12));
            await routeCache.save(cacheKey, result);
            return result;
        } catch (e) {
            // Tier 3: Fallback to Haversine
            return _computeHaversineRoute(request);
        }
    }

    Future<RouteResult> _fetchOSRMRoute(RouteRequest request) async {
        final url = Uri.parse(
            'https://router.project-osrm.org/route/v1/driving/'
            '${request.origin.lng},${request.origin.lat};'
            '${request.destination.lng},${request.destination.lat}'
            '?geometries=geojson&steps=true&overview=full'
        );

        final response = await httpClient.get(url);
        if (response.statusCode == 200) {
            final json = jsonDecode(response.body);
            final route = json['routes'][0];

            return RouteResult(
```

```
        origin: request.origin,
        destination: request.destination,
        distance: (route['distance'] / 1000).toDouble(), // meters to km
        duration: (route['duration'] / 60).toDouble(), // seconds to minutes
        polyline: route['geometry']['coordinates'],
        steps: route['legs'][0]['steps'].map((s) => _parseStep(s)).toList(),
        isFallback: false,
    );
}
throw Exception('OSRM request failed');
}

RouteResult _computeHaversineRoute(RouteRequest request) {
    final distance = _haversineDistance(
        request.origin.lat, request.origin.lng,
        request.destination.lat, request.destination.lng,
    );

    final speedKmH = _getTravelModeSpeed(request.travelMode);
    final duration = distance / speedKmH * 60; // km / (km/h) * 60 = minutes

    return RouteResult(
        origin: request.origin,
        destination: request.destination,
        distance: distance,
        duration: duration,
        polyline: [
            [request.origin.lng, request.origin.lat],
            [request.destination.lng, request.destination.lat],
        ],
        steps: [],
        isFallback: true,
    );
}
```

4.4 Chatbot Implementation

The initial chatbot prototype used a knowledge base and simple intent patterns stored in JSON assets. The following simplified example illustrates the early design rather than

the complete final implementation:

```
{
  "intents": [
    {
      "intent": "ask_best_season",
      "patterns": [
        "what is the best time to visit",
        "when should i go",
        "best season for"
      ],
      "response_template": "The best time to visit {destination} is {season}."
    },
    {
      "intent": "ask_budget",
      "patterns": [
        "how much budget",
        "cost for",
        "price of visiting"
      ],
      "response_template": "{destination} is suitable for {budget_level}
travelers."
    },
    {
      "intent": "emergency",
      "patterns": [
        "injured", "lost", "emergency", "help", "police", "ambulance"
      ],
      "response_template": "Emergency assistance may be required. Show verified
Nepal emergency contacts and the user's available location."
    }
  ],
  "destinations_knowledge": [
    {
      "name": "Ghandruk",
      "facts": [
        "A scenic Gurung village in the Annapurna region",
        "Known for hospitality and homestays",
        "Traditional stone houses and terraced fields",
        "Popular for trekking and cultural tourism"
      ]
    }
  ]
}
```

```
]
}
```

The early prototype processed user input using deterministic emergency checks, intent matching, entity extraction, and template generation:

```
class ChatbotService {
    final ChatbotKnowledge knowledge;

    Future<String> processUserInput(String userInput) async {
        // Emergency detection
        if (_isEmergency(userInput)) {
            return _generateEmergencyResponse();
        }

        // Intent classification
        final intent = _classifyIntent(userInput);

        // Entity extraction
        final entities = _extractEntities(userInput, intent);

        // Response generation
        final response = _generateResponse(intent, entities);

        return response;
    }

    String _classifyIntent(String input) {
        final lowerInput = input.toLowerCase();
        for (final intent in knowledge.intents) {
            for (final pattern in intent.patterns) {
                if (lowerInput.contains(pattern)) {
                    return intent.intent;
                }
            }
        }
        return 'unknown';
    }

    Map<String, String> _extractEntities(String input, String intent) {
        final entities = <String, String>{};

        // Extract destination name
```

```
for (final dest in knowledge.destinations) {
    if (input.contains(dest.name)) {
        entities['destination'] = dest.name;
    }
}

return entities;
}
}
```

In the final system, `IntelligenceOrchestrator` initialises the complete offline intelligence stack. It loads the tourism knowledge repository, destination and accommodation gazetteers, intent examples, semantic embeddings, BM25 index, translation resources, and emergency contacts. Each user message passes through multilingual normalisation, tokenisation, stemming, stop-word removal, named-entity recognition, synonym expansion, hybrid intent classification, dialogue-state tracking, retrieval reranking, and response generation. The safety layer is checked before ordinary tourism logic, and every response path records the completed dialogue turn so that conversation memory remains consistent.

When the backend is available, the chatbot can request online enhancement through the actual `/chat` endpoint. The local answer and local state remain available if that request fails:

```
Future<String> enhanceWithLLM(String userInput, String offlineResponse) async {
    try {
        final response = await http.post(
            Uri.parse('${backendUrl}/chat'),
            body: jsonEncode({
                'question': userInput,
                'language': 'en',
                'top_k': 5,
                'history': recentHistory,
            }),
            headers: {'Content-Type': 'application/json'},
        ).timeout(Duration(seconds: 5));

        if (response.statusCode == 200) {
            final data = jsonDecode(response.body);
            return data['answer'] ?? data['reply'];
        }
    } catch (e) {
        // Timeout or network error; use offline response
    }
}
```

```
    return offlineResponse;
}
```

4.5 Recommendation Engine

The first mobile recommendation prototype scored destinations through direct attribute matching and diversity rules. The simplified listing below represents that development stage and explains the foundation described in the interim submission:

```
class RecommendationEngine {
    final List<Destination> allDestinations;

    List<Destination> generateRecommendations(UserPreferences prefs) {
        // Score each destination
        final scored = allDestinations.map((dest) {
            double score = 0.0;

            // Category matching
            if (prefs.preferredCategories.contains(dest.category)) {
                score += 10;
            }

            // Activity matching
            final matchingActivities = dest.activities
                .where((a) => prefs.preferredActivities.contains(a))
                .length;
            score += matchingActivities * 2;

            // Budget alignment
            if (dest.budgetLevel == prefs.budgetLevel) {
                score += 5;
            }

            // Season appropriateness
            final currentMonth = DateTime.now().month;
            final currentSeason = _getSeasonFromMonth(currentMonth);
            if (dest.bestSeason.contains(currentSeason)) {
                score += 5;
            }
        });
    }
}
```

```
// Adventure level
if ((dest.adventureLevel - prefs.adventureLevel).abs() <= 1) {
    score += 3;
}

// Family-friendly preference
if (prefs.travelingWithFamily && dest.familyFriendly) {
    score += 5;
}

return (destination: dest, score: score);
}).toList();

// Sort by score and apply diversity
scored.sort((a, b) => b.score.compareTo(a.score));

final recommendations = <Destination>[];
final categoriesSeen = <String>{};

for (final entry in scored) {
    // Ensure diversity: don't recommend too many from same category
    if (!categoriesSeen.contains(entry.destination.category) ||
        categoriesSeen.length < scored.length / 3) {
        recommendations.add(entry.destination);
        categoriesSeen.add(entry.destination.category);
    }

    if (recommendations.length >= 10) break;
}

return recommendations;
}
}
```

The final backend recommendation pipeline is substantially more advanced. It builds a natural-language query from activity, budget, season, vibe, family, and adventure preferences. A Sentence-BERT encoder retrieves semantically related destinations from a normalised embedding index. Candidate scores are blended with structured activity/category signals before collaborative and popularity information is introduced. Cold-start requests deliberately suppress collaborative weight and use a limited popularity prior; warm-user requests give greater weight to interaction-derived collaborative scores. The

contextual reranker then assesses activity, vibe, season, budget, accessibility, family suitability, adventure preference, and accommodation fit. It also applies district and category diversity constraints.

The final ranking stage uses a LightGBM LambdaRank model when a trained model is available. The ranker consumes semantic, collaborative, contextual, popularity, activity, season, budget, and vibe features. Before sufficient interaction data exists, it applies a deterministic fallback formula of 0.50 semantic, 0.20 collaborative, and 0.30 contextual score. The response includes component scores and natural-language recommendation reasons. This makes the result more transparent than a single unexplained match value and directly supports the “Why recommended” panel and score breakdown shown by the Flutter interface.

The offline Flutter recommender uses precomputed destination embeddings, contextual components, local affinity signals, constraint penalties, and diversity limits. Therefore, losing the backend does not reduce recommendation behaviour to a static popularity list. Instead, the app continues to produce personalised results from its bundled data while cached recommendations and queued interactions preserve continuity.

4.6 Destination Data Curation

The destination dataset comprises 300 curated entries for Gandaki Province, supported by 771 accommodation records and 300 local WebP destination images. Each destination was researched using:

- Wikipedia articles on Gandaki villages and tourism sites.
- OpenStreetMap geographic data.
- Nepal Tourism Board official resources.
- Local tourism guides and travel blogs.
- Community input and field knowledge.

Each destination record includes 15+ attributes: name, description, coordinates, category, activities, best season, budget level, accessibility, adventure level, family-friendliness, cultural richness, nature score, tags, and image reference.

table 4.2 shows sample destination records.

4.7 Key Application Screens

The application includes 11 primary screens:

Name	Category	Activities	Budget	Season	Family
Ghandruk	Village	Trekking, Culture	Budget	Aut, Spr	Yes
Ghorepani	Trekking	Hiking, Sunrise	Budget	Aut, Spr	Yes
Pokhara	City	Paragliding, Lake	Moderate	Year	Yes
Bandipur	Historic	Walking, Culture	Budget	Aut, Spr	Yes
Sikles	Village	Trekking, Nature	Budget	Aut, Spr	Yes

Table 4.2: Sample destination records from the Gandaki dataset. Each destination includes 15+ attributes for filtering and recommendation.

4.7.1 Home Screen

Displays destination discovery, recommendations, search access, and quick-access widgets for saved places and recent activity.

4.7.2 Destination Browsing

Displays destinations as cards with photos, names, ratings, and categories. Supports filtering by category, budget, activity, season, and adventure level.

4.7.3 Destination Detail

Shows comprehensive information for a selected destination, including description, activities, best season, budget, accommodations, gallery, map location, and save button.

4.7.4 Map Screen

Interactive map showing destinations as markers, route display, user location, map controls, and offline status.

4.7.5 Chatbot Screen

Chat interface for asking tourism questions, showing message history, typing indicator, and offline/online mode indication.

4.7.6 Recommendations Screen

Displays personalized destination recommendations based on user preferences.

4.7.7 Saved Places Screen

Shows destinations saved by the user, with removal options and quick navigation.

4.7.8 Account/Preferences Screen

User profile, preference settings, saved destinations, trip history, and account management.

4.7.9 Settings Screen

App settings including map style, language, offline map status, analytics preferences, and app information.

4.7.10 Onboarding Screen

Initial app experience introducing features, permission requests (location, storage), and preference selection.

4.7.11 About Screen

Project information, team credits, open-source acknowledgements, and links.

Chapter 5

Evaluation

5.1 Evaluation Methodology

The evaluation of Paila Nepal addresses four dimensions:

1. **Offline Functionality:** Testing that core features (destination display, map rendering, routing fallback, chatbot responses) operate correctly without network connectivity.
2. **Recommendation Quality:** Measuring ranked retrieval with Precision@10, Recall@10, nDCG@10, and MRR across five tourism scenarios.
3. **Software Verification and Performance:** Running static analysis, backend tests, Flutter tests, and focused performance checks, while recording failures rather than suppressing them.
4. **Usability:** Preparing a System Usability Scale (SUS) instrument and identifying the limitations created by the absence of a completed formal participant study.

These dimensions provide triangulation rather than relying on one favourable measure. Manual offline checks establish whether the intended traveler workflows remain available; automated checks expose code and lifecycle defects; recommendation metrics measure ranking behaviour; asset inspection quantifies the distribution cost of local capability; and the usability discussion limits claims that cannot yet be supported by participant evidence. The evaluation therefore distinguishes implemented capability, measured quality, and remaining uncertainty. This distinction is important because an offline-first prototype can appear successful during a short demonstration while still containing weaknesses in ranking coverage, test isolation, performance consistency, accessibility, or real-world field reliability under realistic conditions.

5.2 Offline Functionality Testing

Testing was conducted by launching the application in offline mode (via airplane mode or connectivity disabling) and verifying that:

1. Application launches successfully.
2. Home screen displays without network errors.
3. Destination list loads all 300 destinations from local JSON assets.
4. Destination cards display photographs from local WebP assets.
5. Map screen renders with local MBTiles tiles at zoom levels 7–12.
6. Route computation falls back to Haversine formula when OSRM is unavailable.
7. Chatbot processes user input and generates responses using local knowledge base.
8. Saved destinations persist in SQLite and remain accessible.
9. Recommendations can be generated from local cached preferences and destination data.
10. No unhandled exceptions occur during normal usage patterns.

The development-stage manual checks recorded in Table 5.1 indicate that the intended offline paths were available. These checks are useful evidence of feature integration, but they should not be treated as a replacement for a controlled multi-device field study.

Feature	Online Mode	Offline Mode	Result
App Launch	Yes	Yes	Pass
Home Screen	Yes	Yes	Pass
Destination List	Yes	Yes	Pass
Destination Detail	Yes	Yes	Pass
Map Display	Yes	Yes	Pass
Route (OSRM)	Yes	No	Online only
Route (Fallback)	Yes	Yes	Pass
Chatbot (Offline)	Yes	Yes	Pass
Chatbot (LLM)	Yes	No	Graceful degradation
Saved Destinations	Yes	Yes	Pass
Recommendations	Yes	Yes	Pass
Image Display	Yes	Yes	Pass

Table 5.1: Development-stage offline functionality checks. OSRM and online LLM enhancement are intentionally unavailable offline, while local fallbacks preserve core functionality.

5.3 Software Verification and Performance

5.3.1 Automated Verification Results

The final verification pass used commands that can be repeated from the project repository. The results are deliberately reported with their failures because a critical evaluation is more useful than an unsupported claim that every test passes.

Verification	Result	Interpretation
<code>flutter analyze -no-pub</code>	Pass	No static-analysis issues were reported.
<code>pytest backend/tests -q</code>	Pass	Nine backend tests passed.
<code>flutter test -no-pub</code>	Partial failure	Eighteen tests passed before the recommendation-tab widget test exposed a pending backend-health timer after disposal.
Focused intelligence performance test	Partial failure	Deterministic emergency detection passed its target, but warm offline translation took 570 ms and exceeded the 500 ms target.
<code>pytest evaluation -q</code>	No tests discovered	The evaluation package contains benchmark code but no pytest-discoverable tests.

Table 5.2: Repeatable automated verification results from the final report pass.

The static-analysis and backend-test results indicate that the core codebase is structurally sound and that the tested backend services behave as expected. However, the widget-test failure reveals a lifecycle issue: the recommendation screen starts a backend health check that remains active after the test disposes the widget tree. This does not demonstrate a user-facing crash, but it weakens test isolation and should be corrected by injecting or cancelling the timer. The translation performance failure is similarly informative. A 570 ms warm offline response remains usable, but it exceeds the project’s defined target and suggests that asset loading, matching, or initialisation should be profiled and cached more aggressively.

5.3.2 APK Size Analysis

The latest release APK in the project build output is approximately 94.97 MB. The largest deliberately bundled offline asset groups include approximately 8.74 MB of local WebP destination images, an 8.17 MB MBTiles map package, a 1.38 MB destination JSON file, and a 0.40 MB accommodation JSON file. The remaining size includes the Flutter runtime, native libraries, plugins, fonts, code, and other resources. This size is a direct trade-off: a smaller network-dependent app would be easier to distribute, whereas

the current package protects functionality after connectivity is lost. Future optimisation should focus on Android App Bundles, architecture splits, image quality experiments, and downloadable regional map packs.

5.4 Quantitative Evaluation Results

The recommendation system was evaluated across five tourism scenarios using graded relevance labels from zero to three. A destination with a relevance grade of at least two was treated as relevant for Precision@10, Recall@10, and MRR, while nDCG@10 retained ranking-sensitive graded relevance. Precision measures how many returned destinations are relevant; recall measures how many of all relevant destinations are recovered; nDCG rewards placing highly relevant destinations near the top; and MRR records the reciprocal rank of the first relevant result (Järvelin and Kekäläinen, 2002).

Scenario	P@10	R@10	nDCG@10	MRR
Cultural trekking	0.8000	0.3333	0.8305	1.0000
High adventure	0.7000	0.4375	0.6797	1.0000
Budget relaxation	0.5000	0.5000	0.6152	1.0000
Family-friendly cultural	0.8000	0.3333	0.8701	1.0000
Pilgrimage route	0.6000	0.6000	0.7759	1.0000
Average	0.6800	0.4408	0.7543	1.0000

Table 5.3: Recommendation-quality results across the five dissertation scenarios.

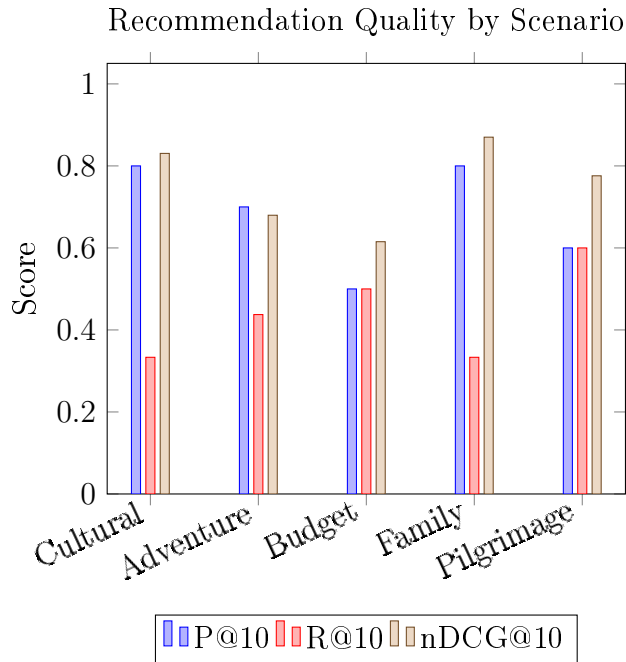


Figure 5.1: Precision, recall, and ranking quality across five tourism scenarios.

The results show that the recommender is strongest for cultural trekking and family-friendly cultural queries. These scenarios achieve Precision@10 of 0.8000 and nDCG@10 above 0.83, suggesting that explicit cultural, family, activity, and season attributes are represented effectively in the dataset and ranking features. Budget relaxation is the weakest scenario, with Precision@10 of 0.5000 and nDCG@10 of 0.6152. This result is understandable because concepts such as peacefulness, relaxation, low cost, and proximity to suitable accommodation are less precisely represented than explicit categories such as pilgrimage or cultural heritage. Improving accommodation pricing, tranquillity features, and budget evidence would therefore be more valuable than simply changing model weights.

MRR is 1.0000 for every scenario, meaning that the first returned destination was relevant in all five tests. This is important for mobile interfaces because users frequently inspect only the first few results. Nevertheless, MRR should not be interpreted as evidence that the complete ranking is perfect. Average Recall@10 is only 0.4408, demonstrating that more than half of all destinations labelled relevant were not recovered in the first ten results. The results therefore support a balanced conclusion: the system is effective at placing at least one useful answer first, but candidate retrieval, attribute completeness, and coverage still require improvement.

5.5 Objective Traceability and Evaluation Validity

The evaluation evidence can be traced directly to the project objectives. Offline launch, local destination loading, WebP image display, and MBTiles rendering demonstrate that the application preserves its core discovery experience without a network. The routing checks exercise both the online OSRM path and the Haversine fallback. Local chatbot and translation tests examine the offline intelligence objective, while the five recommendation scenarios evaluate semantic retrieval, contextual ranking, and explanation-oriented personalisation. SQLite and Hive behaviour is exercised through saved-place, review, cache, preference, and interaction workflows. Finally, static analysis, backend tests, Flutter tests, APK inspection, and manual offline checks assess the system as an integrated software product rather than as a collection of isolated algorithms.

This traceability also reveals where evidence is weaker. The recommendation metrics show ranking behaviour for defined scenarios, but they do not prove that every tourist would judge relevance in the same way. The relevance labels and scenarios reflect the project's domain assumptions, so changes to labels could change Precision, Recall, nDCG, and MRR. Similarly, a passing offline check proves that a path worked during development; it does not establish long-term reliability across all Android versions, memory constraints, or unusual user actions. The absence of a completed SUS participant study means that ease of use, accessibility, and trust cannot be inferred from functional tests alone.

Several threats to validity therefore remain. Internal validity is affected by the inter-

action between cached state, device performance, and test order. For example, a warmed recommendation or translation path can appear faster than a first launch. Construct validity is limited because ranking metrics measure relevance and ordering, not whether a recommendation leads to a safe or satisfying visit. External validity is limited by the focus on Gandaki Province, a single Android-oriented implementation, five evaluation scenarios, and development hardware. Ecological validity is also constrained because airplane-mode testing cannot reproduce every rural condition, such as intermittent connectivity, low battery, inaccurate GPS, or rapidly changing road access.

Reproducibility was improved by recording the commands, dataset sizes, asset sizes, metric definitions, scenario-level results, and observed failures. Future evaluation should run the same checks on multiple physical devices, distinguish cold and warm performance, simulate intermittent rather than simply absent connectivity, and recruit domestic and international travelers for task-based testing. Recommendation labels should be reviewed by tourism-domain participants, and online experiments should compare the semantic baseline, contextual reranker, and learned ranking components. These steps would convert the present development evidence into stronger claims about reliability, usability, and real-world tourism value.

5.6 Usability Evaluation

The System Usability Scale (SUS) was selected as the intended formal usability instrument because it provides a concise ten-item questionnaire that can be applied across different interactive systems (Brooke, 1996). The questionnaire is retained in Table 5.4 so that a later participant study can use a consistent instrument. However, a completed formal SUS study with a defensible participant sample was not available at the time of this report. Consequently, no SUS score is reported, and usability conclusions are limited to development feedback, static analysis, widget testing, and observation of the implemented interaction flows.

This limitation matters because functional correctness does not guarantee that tourists with different languages, ages, technical confidence, or accessibility needs will find the interface easy to use. A future study should recruit both domestic and international travellers, ask them to complete offline discovery, map, chatbot, translation, save, and recommendation tasks, and combine SUS scores with task completion rates and qualitative interviews.

5.7 Comparison with Related Work

table 5.5 compares Paila Nepal with related tourism applications on key features.

Paila Nepal’s distinctive contributions are:

Item	Statement
1	I think that I would like to use this system frequently.
2	I found the system unnecessarily complex.
3	I thought the system was easy to use.
4	I think that I would need the support of a technical person to use this system.
5	I found the various functions in this system were well integrated.
6	I thought there was too much inconsistency in this system.
7	I would imagine that most people would learn to use this system very quickly.
8	I found the system very cumbersome to use.
9	I felt very confident using the system.
10	I needed to learn a lot of things before I could get going with this system.

Table 5.4: System Usability Scale (SUS) items used for usability evaluation.

1. **Rural Gandaki Specialisation:** Focuses specifically on rural Gandaki Province destinations, rather than global coverage.
2. **Truly Offline-First:** All core features function without network connectivity, rather than relying on online connectivity as primary.
3. **Local Chatbot:** Provides offline conversational tourism assistance without depending on cloud LLMs.
4. **Integrated Offline Map + Routing + Recommendations:** Combines multiple offline systems into a single cohesive application.
5. **Tourism-Specific Design:** Includes tourism-relevant attributes (activities, seasons, budget levels, family-friendliness) in destination models.

5.8 Limitations and Critical Analysis

Despite the successful implementation of offline-first architecture, several limitations should be acknowledged:

5.8.1 Limited Destination Dataset

The application includes 300 curated Gandaki destinations. While this provides meaningful coverage of major tourism sites, the dataset may not include every lesser-known destination, homestay, or emerging tourism point of interest. Additionally, destination

Feature	Google Maps	Trip Advisor	OS MAnd	Paila Nepal
Offline Maps	•	Not supported	•	••
Gandaki Focus	Not supported	Not supported	Not supported	••
Chatbot Assistance	Not supported	Partial	Not supported	•
Destination Recommendations	Partial	•	Not supported	•
Offline Routing	Not supported	Not supported	•	•
Saved Destinations	•	•	Not supported	•
Offline Image Display	Not supported	Not supported	•	••
Local NLP Chatbot	Not supported	Not supported	Not supported	•
Offline-First Design	Not supported	Not supported	Partial	••
Tourism Specialisation	Not supported	•	Not supported	••

Table 5.5: Feature comparison of tourism applications. Legend: •• = full support, • = partial support, Not supported = not supported. Paila Nepal uniquely combines offline-first architecture with rural Gandaki tourism specialisation.

information is static; it does not update in real-time based on community input, seasonal changes, or recent events.

5.8.2 Limited Offline Map Zoom

The offline map MBTiles includes zoom levels 7–12, suitable for regional browsing and destination-level orientation. However, users who zoom beyond level 12 see scaled-up tiles or blank areas if online fallback is disabled. Full street-level mapping would require significantly larger map tiles (potentially 500+ MB), exceeding practical APK distribution constraints.

5.8.3 Approximate Offline Routing

When OSRM is unavailable, the Haversine straight-line route provides direction and distance estimation but not turn-by-turn navigation or road-aware routing. Users must supplement with local knowledge or request directions from residents for final navigation. A fully offline turn-by-turn system would require bundling a complete OpenStreetMap road network, which is beyond current scope.

5.8.4 Limited Collaborative Data

The application includes local ratings/reviews and logs recommendation-related interactions, with queued batch synchronisation to the backend. However, the project has not accumulated a large, representative population of real users. Collaborative scores and learning-to-rank training are therefore constrained by sparse or synthetic interactions. The architecture is ready to learn from future usage, but current evaluation should not

imply that the collaborative component has already captured stable population preferences.

5.8.5 Language Limitations

The offline intelligence system supports English, Devanagari Nepali, and romanised Nepali through normalisation, phrasebooks, templates, glossaries, and local retrieval. Nevertheless, it is not equivalent to a large multilingual neural model. Unseen grammar, spelling variation, long-form translation, dialect differences, and culturally specific expressions can still reduce accuracy. The focused performance test also showed a 570 ms warm offline translation result against a 500 ms target, indicating room for optimisation.

5.8.6 Device Storage Dependency

The offline-first design requires bundling significant assets, resulting in a current release APK of approximately 94.97 MB. Older Android devices with limited storage or expensive data plans may find the initial download difficult. This is a reasonable but non-trivial trade-off for comprehensive offline functionality.

5.8.7 Lack of Real-Time Data

The app does not provide real-time information about weather, current events, road conditions, or accommodation availability. This would require continuous backend connectivity to be meaningful.

5.9 Societal Impact and Contribution

Paila Nepal demonstrates a potential contribution to rural tourism in Nepal:

5.9.1 Information Accessibility

By bundling comprehensive destination information locally, the application increases information accessibility in rural areas where internet connectivity is unreliable. Travelers have access to structured, contextual destination information regardless of network state.

5.9.2 Economic Potential

By increasing digital visibility of rural Gandaki destinations, the application may contribute to tourism demand for rural accommodations, local guides, artisan goods, and cultural experiences. This potential economic benefit accrues primarily to rural communities.

5.9.3 Offline-First Design Pattern Contribution

Paila Nepal demonstrates how offline-first architecture can enhance accessibility in low-connectivity contexts. The design patterns and lessons learned may inform the development of similar applications for other rural tourism regions or resource-constrained contexts globally.

5.9.4 Open-Source Potential

The project (including code, dataset, and architecture) could be open-sourced, allowing other regions to adapt the platform for their local tourism contexts.

5.10 Ethical Considerations

Several ethical considerations were addressed during development:

5.10.1 Data Privacy

User location data (if shared) is stored locally on the device. The application does not transmit location data to the backend without explicit user consent. Chatbot interactions remain on-device in offline mode; online LLM requests require user acknowledgement.

5.10.2 Accessibility

The application was designed with accessibility in mind, including readable font sizes, high-contrast colors, and support for standard Android accessibility services. However, some complex features (maps, routing) may benefit from additional accessibility enhancements.

5.10.3 Cultural Sensitivity

Destination descriptions and recommendations respect local culture and traditions. The application emphasizes cultural tourism as meaningful exchange rather than exploitation, and includes warnings about respectful conduct in sacred sites.

5.10.4 Fair Attribution

Destination photographs are sourced from Wikimedia Commons, Wikipedia, and openly-licensed sources. Attribution is provided where appropriate. Destination descriptions are either original or paraphrased from openly-available sources.

Chapter 6

Challenges and Solutions

6.1 Map Tile Format and Compatibility

Challenge: Determining the appropriate map tile format (raster vs. vector) and ensuring compatibility with Flutter map libraries.

Solution: Selected raster PNG tiles in MBTiles format. Raster tiles are directly renderable as images, compatible with `flutter_map`, and suitable for offline distribution. Vector tiles (PBF format) would require client-side rendering, adding complexity and library dependencies.

6.2 MBTiles Coordinate System

Challenge: Resolving coordinate system differences between TMS (Tile Map Service) and XYZ tile coordinate conventions.

Solution: Validated the MBTiles file against both coordinate systems. The Flutter MBTiles provider handles coordinate transformation automatically, so the issue was identified and resolved during testing.

6.3 APK Size Management

Challenge: The combination of the Flutter runtime, destination images, map tiles, and local data resulted in a large APK.

Solution: Optimised images to WebP format, limited map tile zoom levels to 7–12, and limited the destination dataset to Gandaki Province. These choices balanced offline functionality with a release APK size of approximately 94.97 MB.

6.4 Offline Routing Fallback

Challenge: Providing useful route guidance when online OSRM routing is unavailable.

Solution: Implemented the three-tier routing architecture: cache to OSRM to Haversine fallback. The Haversine fallback provides direction and distance estimation, suitable for initial navigation planning.

6.5 Vector vs. Raster Tile Performance

Challenge: Vector tiles offer better scalability and smaller file sizes but require client-side rendering.

Solution: Chose raster tiles for simplicity and compatibility with available Flutter libraries. Raster tiles are directly renderable, requiring no client-side vector processing. This trade-off prioritised compatibility over file size.

Chapter 7

Conclusion

7.1 Summary of Achievements

Paila Nepal successfully demonstrates how offline-first mobile architecture can enhance digital accessibility and tourism promotion in rural contexts with limited internet connectivity. The project achieves the following:

1. **Offline-First Android Application:** A working Flutter-based Android prototype that preserves destination discovery, map access, fallback route guidance, recommendations, and chatbot assistance without requiring continuous internet connectivity.
2. **Comprehensive Destination Dataset:** 300 curated Gandaki Province destinations with more than 20 structured and descriptive attributes, 771 accommodation records, and 300 local images.
3. **Local Asset Integration:** Bundled 300 destination photographs (WebP format), offline map tiles (MBTiles), and chatbot knowledge bases, enabling rich offline experiences.
4. **Three-Tier Routing System:** Route guidance that gracefully degrades from OSRM road-aware routing to cached routes to Haversine approximation, ensuring users always receive directional guidance.
5. **Offline Intelligence:** Local multilingual processing, hybrid intent classification, knowledge retrieval, dialogue memory, safety handling, translation, and optional online LLM enhancement.
6. **Persistent User Data:** SQLite, Hive, and SharedPreferences support for saved destinations, reviews, recommendation caches, route cache, preferences, and queued interactions.

7. **Explainable Hybrid Recommendations:** SBERT retrieval, collaborative/popularity signals, contextual reranking, cold-start handling, diversity filtering, ML rescoring, and human-readable recommendation reasons.
8. **Evidence-Based Evaluation:** Five recommendation scenarios, repeatable software checks, explicit reporting of remaining test failures, and a critical analysis of limitations.

7.2 Contribution to Knowledge

This project contributes to academic and practical understanding in several areas:

7.2.1 Offline-First Architecture for Tourism

The three-tier routing architecture, tiered data availability, and graceful degradation patterns provide a practical reference for other tourism and low-connectivity applications.

7.2.2 Rural Tourism Digitalisation

The focus on rural Gandaki demonstrates how local content, accessibility, and personalisation can support rural tourism visibility.

7.2.3 Mobile Application Design for Low-Connectivity Contexts

The architecture also illustrates mobile design choices applicable to other regions with unreliable connectivity.

7.2.4 Open Dataset and Methodology

The dataset, knowledge resources, and architecture could be adapted for other regional tourism contexts.

7.3 Future Work

Several directions could extend Paila Nepal:

7.3.1 Real User Feedback and Iteration

Deploying the application to real travelers in Gandaki Province, collecting usage data, and iterating based on user feedback would validate and refine the design.

7.3.2 Expanded Destination Dataset

Incorporating additional destinations beyond the current 300, including homestays, emerging tourism points, and user-contributed information.

7.3.3 Full-Length Offline Routing

Integrating a local road network (e.g., PBF OpenStreetMap extract) to enable turn-by-turn offline navigation without depending on cached routes.

7.3.4 Expanded Multilingual Support

Extending the existing English, Devanagari Nepali, and romanised Nepali resources with broader vocabulary, stronger neural translation, Hindi, Mandarin, and additional tourism-specific language coverage.

7.3.5 Community Contributions

Implementing a backend system for users to submit destination photos, reviews, and information updates, with moderation and synchronisation to the offline app.

7.3.6 iOS Support

Extending the application to iOS using Flutter’s cross-platform capabilities.

7.3.7 Recommendation Learning from Real Users

The current system already contains collaborative filtering, semantic embeddings, contextual reranking, a LambdaRank training path, and experimental two-tower components. Future work should therefore focus on collecting consented real-user interactions, evaluating learned ranking against the semantic-only baseline, monitoring fairness and coverage, and retraining safely from representative data.

7.3.8 Integration with Nepal Tourism Services

Partnering with Nepal Tourism Board, accommodation networks, and local tourism operators to integrate booking, weather, verified transport updates, and real-time availability information.

7.4 Reflection on Project Outcomes

Paila Nepal addresses the research question by combining bundled destination data and maps with route fallback, local intelligence, offline recommendation assets, and persistent local state. Online-only features, including OSRM road routes and language-model enhancement, deliberately degrade to local alternatives.

However, the project also highlights inherent trade-offs in offline-first design. Bundling significant assets (images, maps) results in a larger APK. Static offline data cannot reflect real-time changes in availability, weather, or events. Offline route computation is approximate compared to road-aware online routing. These trade-offs are intentional and reflect the priorities of accessibility and offline reliability over comprehensiveness and currency.

The evaluation shows both promise and incompleteness. MRR of 1.0000 demonstrates that every tested scenario placed a relevant destination first, while average Recall@10 of 0.4408 shows that relevant destinations are still missed. Static analysis and backend tests pass, but the remaining Flutter widget-test timer and translation performance target demonstrate that further engineering work is required. This balanced outcome is more valuable than claiming production readiness: the project establishes a substantial and testable foundation for future tourism applications and rural digital development initiatives.

7.5 Final Remarks

Paila Nepal demonstrates that tourism technology can respect connectivity constraints by treating offline operation as a core requirement. Its practical patterns, documented limitations, and extensible architecture provide a foundation in practice for future systems serving Gandaki Province and other low-connectivity regions.

References

- Adomavicius, Gediminas and Alexander Tuzhilin (2005). “Toward the next generation of recommender systems: A survey of the state-of-the-art and possible extensions”. In: *IEEE Transactions on Knowledge and Data Engineering* 17.6, pp. 734–749.
- Brooke, John (1996). “SUS: A quick and dirty usability scale”. In: *Usability Evaluation in Industry*. Ed. by Patrick W. Jordan et al. Taylor and Francis, pp. 189–194.
- Burke, Robin (2002). “Hybrid recommender systems: Survey and experiments”. In: *User Modeling and User-Adapted Interaction* 12.4, pp. 331–370.
- FastAPI (2026). *FastAPI documentation*. URL: <https://fastapi.tiangolo.com/> (visited on 06/04/2026).
- Flutter (2026). *Flutter documentation*. URL: <https://flutter.dev/> (visited on 06/04/2026).
- Järvelin, Kalervo and Jaana Kekäläinen (2002). “Cumulated gain-based evaluation of information retrieval techniques”. In: *ACM Transactions on Information Systems* 20.4, pp. 422–446.
- Nepal Tourism Board (2023). *Nepal Tourism Statistics 2023*. URL: <https://trade.ntb.gov.np/downloads/nepal-tourism-statistic-2023/> (visited on 06/04/2026).
- Open Source Routing Machine Project (2026). *OSRM API documentation*. URL: <https://project-osrm.org/docs/> (visited on 06/04/2026).
- OpenStreetMap Foundation (2026). *OpenStreetMap*. URL: <https://www.openstreetmap.org/> (visited on 06/04/2026).
- Reimers, Nils and Iryna Gurevych (2019). “Sentence-BERT: Sentence embeddings using Siamese BERT-networks”. In: *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing*, pp. 3982–3992.
- Salton, Gerard and Christopher Buckley (1988). “Term-weighting approaches in automatic text retrieval”. In: *Information Processing and Management* 24.5, pp. 513–523.
- SQLite (2026). *About SQLite*. URL: <https://www.sqlite.org/about.html> (visited on 06/04/2026).
- Tkalčič, Marko and Li Chen (2019). “Personality and context adaptive point of interest recommendation”. In: *ACM Transactions on Intelligent Systems and Technology (TIST)* 10.4, pp. 1–28.

Ye, Mao, Peifeng Yin, and Wang-Chien Lee (2010). “Location recommendation for location-based social networks”. In: *Proceedings of the 18th SIGSPATIAL International Conference on Advances in Geographic Information Systems*, pp. 458–461.

Appendix A

Sample Destination Data

table [A.1](#) shows the first five destinations from the curated dataset in JSON structure.

Field	Example Value
id	destination_001
name	Ghandruk
description	A scenic Gurung village in the Annapurna foothills with traditional stone houses, terraced fields, and warm hospitality...
latitude	28.3956
longitude	83.8622
category	village
activities	["trekking", "village_tour", "culture", "nature"]
best_season	["autumn", "spring"]
budget_level	budget
accessibility	moderate
adventure_level	2
family_friendly	true
cultural_richness	8
nature_score	7

Table A.1: Sample destination data structure showing attributes included for each of 300 Gandaki Province destinations.

Appendix B

API Endpoint Specifications

The FastAPI backend provides the following endpoints:

Endpoint	Method	Purpose
/health	GET	Health check
/recommend	POST	Hybrid ranked destinations with scores, reasons, cache metadata, and interaction logging
/chat	POST	Online chatbot response with local fall-back support
/chat/offline	POST	Explicit offline rule-based/retrieval response
/translate	POST	Supported-pair translation endpoint
/auth/register, /auth/login, /auth/me	POST/GET	Authentication and current-user access
/destinations,	GET	Destination catalogue and detail access
/destinations/{id}		
/destinations/{id}/accommodations	GET	Accommodation records linked to a destination
/similar/{id}	GET	Similar-destination semantic retrieval
/interactions, /interactions/batch	POST	Single and queued-batch interaction logging
/analytics/recommender	GET	Recommender analytics
/evaluate/	POST	Recommendation evaluation metrics
/evaluate/train-ranker	POST	Train the LightGBM LambdaRank model
/admin/rebuild-index	POST	Rebuild the semantic destination index

Appendix C

Testing Checklist

Testing checklist for offline functionality:

1. Enable airplane mode on test device
2. Launch application and verify successful startup
3. Navigate to home screen and verify no network errors
4. Scroll destination list and verify all 300 destinations load
5. View destination detail screens and verify images display
6. View map screen and verify map tiles render
7. Request route between two destinations and verify Haversine fallback
8. Submit chatbot query and verify offline response
9. Save a destination and verify it persists to SQLite
10. Disable airplane mode and verify app transitions to online mode gracefully
11. Request route and verify OSRM online routing works
12. Clear cache and request previously-used route; verify cache miss and re-computation

Appendix D

Database Schema Diagrams

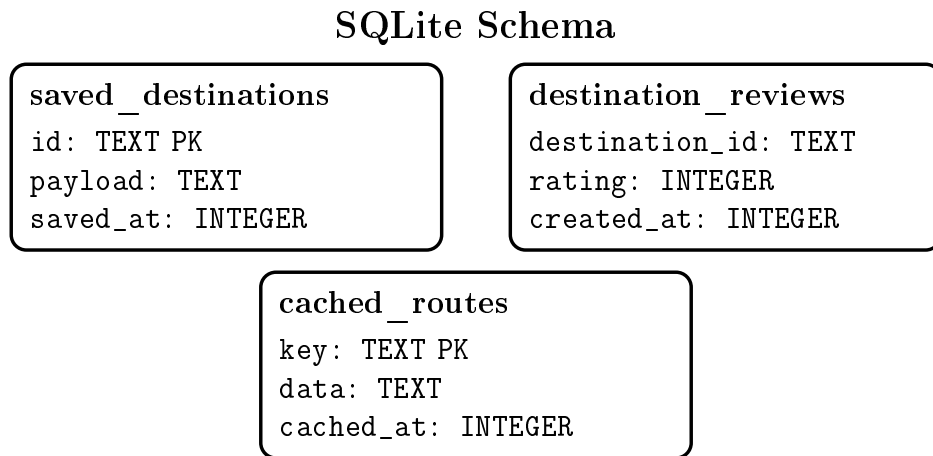


Figure D.1: Simplified SQLite schema for Paila Nepal. The main application database stores saved destinations and reviews, while a separate SQLite database stores the 30-day route cache. Recommendation caches and pending interactions use Hive.

Appendix E

System Requirements

Minimum Device Requirements:

- Android 7.0 (API level 24) or higher
- 2 GB RAM
- 150 MB free storage (for app + data)
- GPS and location services

Recommended Requirements:

- Android 10 or higher
- 4+ GB RAM
- 200+ MB free storage
- WiFi + cellular connectivity options

Development Requirements:

- Flutter SDK 3.0.0+
- Dart 3.x
- Android Studio 4.1+
- Android NDK (for native dependencies)
- Xcode (for iOS development)

Appendix F

Requirements Catalogue and Traceability

This appendix expands the project objectives into assessable requirements. Priority follows the MoSCoW convention: Must, Should, Could, and Won't for the current release. The verification column identifies the strongest available form of evidence.

Table F.1: Functional requirements catalogue.

ID	Requirement	Priority	Evidence
FR-01	The application shall launch and expose its core destination catalogue without an active internet connection.	Must	Airplane-mode scenario
FR-02	The application shall load all 300 curated Gandaki Province destination records from bundled assets.	Must	Dataset validation
FR-03	A user shall be able to search and filter destinations by meaningful tourism attributes.	Must	Widget and manual tests
FR-04	A destination detail view shall present descriptive, geographic, seasonal, activity, and image information.	Must	UI inspection
FR-05	The map shall render bundled Gandaki tiles when online tile services are unavailable.	Must	Airplane-mode scenario
FR-06	The routing service shall prefer a cached route, then an online OSRM route, then an offline Haversine fallback.	Must	Routing branch tests

ID	Requirement	Priority	Evidence
FR-07	Users shall be able to save destinations and retain them between application sessions.	Must	Persistence test
FR-08	Users shall be able to submit and retain local destination reviews.	Should	Persistence test
FR-09	The system shall produce explainable personalized destination recommendations.	Must	Scenario evaluation
FR-10	Recommendation capability shall remain available without the backend.	Must	Offline recommendation test
FR-11	The chatbot shall answer tourism-oriented questions using local intelligence when disconnected.	Must	Intelligence end-to-end test
FR-12	Emergency-like messages shall receive deterministic safety-oriented handling.	Must	Safety scenario test
FR-13	Translation shall support the application's declared English and Nepali travel phrases.	Should	Translation evaluation
FR-14	The application shall queue user interactions locally when the backend is unavailable.	Should	Synchronisation test
FR-15	Queued interactions shall be submitted to the backend after connectivity returns.	Should	Integration scenario
FR-16	The backend shall expose authenticated registration, login, and current user endpoints.	Should	Backend tests
FR-17	The backend shall expose destination and accommodation catalogue endpoints.	Must	API contract test
FR-18	The backend shall expose recommendation, similar-destination, and evaluation endpoints.	Must	API contract test
FR-19	Administrators shall be able to rebuild the semantic destination index.	Could	Endpoint inspection
FR-20	The application shall communicate backend unavailability clearly without preventing local use.	Must	Offline UX inspection

Table F.2: Non-functional requirements catalogue.

ID	Requirement	Priority	Evidence
NFR-01	Core discovery, saved data, local recommendations, maps, and local assistance shall tolerate a complete loss of network connectivity.	Must	Airplane-mode suite
NFR-02	The Android release shall remain practical to distribute despite bundled maps, images, and recommendation assets.	Must	APK size inspection
NFR-03	Personal data and authentication secrets shall not be embedded in public application assets.	Must	Configuration review
NFR-04	Backend passwords shall be stored using a one-way password hashing mechanism.	Must	Auth service test
NFR-05	Recommendation results shall include understandable reasons rather than scores alone.	Should	Response inspection
NFR-06	Ranking evaluation shall use repeatable relevance scenarios and standard information-retrieval metrics.	Must	Evaluation script
NFR-07	The app shall use accessible contrast, readable labels, and touch targets suitable for repeated mobile use.	Should	UI inspection
NFR-08	Failures in online enhancement services shall degrade to local capability rather than crash the application.	Must	Failure injection
NFR-09	Local route cache entries shall expire after a defined freshness period.	Should	Cache inspection
NFR-10	Source modules shall retain clear separation between presentation, domain logic, storage, and remote services.	Should	Architecture review
NFR-11	Automated static analysis shall complete without reported Flutter issues.	Must	<code>flutter analyze</code>
NFR-12	Backend automated tests shall pass in the documented environment.	Must	<code>pytest</code> result
NFR-13	The application shall be maintainable through central configuration of its backend base URL.	Should	Configuration review
NFR-14	Dataset fields shall use a consistent schema that can be validated before packaging.	Must	Dataset validation

ID	Requirement	Priority	Evidence
NFR-15	Safety responses shall be deterministic and shall not depend exclusively on an external language model.	Must	Emergency tests

F.1 Offline Launch and Core Discovery

Identifier	REQ-TRACE-01 / FR-01, FR-02, NFR-01
Purpose	Demonstrate that the application remains useful at initial launch when no network is available.
Preconditions	A release build is installed; airplane mode is enabled; bundled assets have not been removed.
Procedure or decision	Launch the app, dismiss onboarding if required, open the home destination feed, perform a destination search, and open a destination detail view. Repeat after force-closing the application.
Acceptance evidence	The shell loads without a blocking network error; locally packaged destination content is visible; search and detail navigation remain available; the app can be relaunched.
Residual risk	Live events, weather, remote reviews, and newly changed destination information cannot be guaranteed while disconnected.

Traceability note. This record connects a stated project requirement to a concrete implementation or verification activity. It is retained as submission evidence so that the system can be assessed against observable behaviour rather than feature claims alone.

F.2 Bundled Destination Catalogue Integrity

Identifier	REQ-TRACE-02 / FR-02, NFR-14
Purpose	Establish that the packaged catalogue is structurally valid and sufficiently rich for filtering and recommendation.
Preconditions	The destination JSON asset is present and the application's destination model is available.
Procedure or decision	Parse the complete asset; check identifier uniqueness; validate coordinates; confirm required text, category, activity, season, budget, accessibility, family, culture, nature, and image fields; report malformed records.
Acceptance evidence	All 300 intended destination records load through the real model mapping and can be used by discovery and recommendation services.
Residual risk	Curated data can become stale; a future release requires a governed update and validation process.

Traceability note. This record connects a stated project requirement to a concrete implementation or verification activity. It is retained as submission evidence so that the system can be assessed against observable behaviour rather than feature claims alone.

F.3 Offline Map Availability

Identifier	REQ-TRACE-03 / FR-05, NFR-01
Purpose	Confirm that map exploration does not disappear when an online tile provider cannot be reached.
Preconditions	The Gandaki MBTiles asset is packaged; location permission is granted or a known map location is selected.
Procedure or decision	Enable airplane mode, open the map, pan across Pokhara and surrounding Gandaki areas, change supported zoom levels, and reopen the map after restarting the app.
Acceptance evidence	Raster tiles render from the bundled database for the declared coverage and zoom range without a tile-network dependency.
Residual risk	Coverage is deliberately bounded; street-level tiles outside the packaged region or zoom range are unavailable.

Traceability note. This record connects a stated project requirement to a concrete implementation or verification activity. It is retained as submission evidence so that the system can be assessed against observable behaviour rather than feature claims alone.

F.4 Three-Tier Route Resolution

Identifier	REQ-TRACE-04 / FR-06, NFR-08, NFR-09
Purpose	Verify deterministic routing behaviour across cached, online, and fully disconnected states.
Preconditions	Two valid coordinates are selected and the route cache can be inspected or cleared.
Procedure or decision	Request a route once online, repeat to exercise the cache, then clear the relevant entry and request the route in airplane mode. Compare route source, distance, and presentation.
Acceptance evidence	The first eligible online request uses OSRM, the repeated request can use the SQLite cache, and the disconnected cache miss produces a Haversine fallback instead of a crash.
Residual risk	The Haversine line is an estimate and must not be presented as road-aware turn-by-turn navigation.

Traceability note. This record connects a stated project requirement to a concrete implementation or verification activity. It is retained as submission evidence so that the system can be assessed against observable behaviour rather than feature claims alone.

F.5 Persistent Saved Destinations and Reviews

Identifier	REQ-TRACE-05 / FR-07, FR-08
Purpose	Confirm that user-created local state survives navigation and process restarts.
Preconditions	At least one destination is available; application storage has not been manually cleared.
Procedure or decision	Save a destination, submit a local review, navigate away, force-close the app, relaunch, and inspect saved and reviewed content. Then remove the saved item and confirm the change persists.
Acceptance evidence	SQLite-backed saved and review state remains consistent across sessions and reflects create and delete actions.
Residual risk	Device uninstall or storage clearing removes local-only state unless a future synchronisation feature has uploaded it.

Traceability note. This record connects a stated project requirement to a concrete implementation or verification activity. It is retained as submission evidence so that the system can be assessed against observable behaviour rather than feature claims alone.

F.6 Offline Recommendation Continuity

Identifier	REQ-TRACE-06 / FR-09, FR-10, NFR-05
Purpose	Demonstrate that personalization is not exclusively dependent on the FastAPI service.
Preconditions	Destination assets and local recommendation resources are packaged; user preferences can be supplied.
Procedure or decision	Generate recommendations online, disable connectivity, change a preference or context, and generate recommendations again. Inspect ordering, reasons, and absence of blocking errors.
Acceptance evidence	The offline pipeline returns ranked destinations with human-readable reasons and uses locally available semantic and contextual signals.
Residual risk	Offline ranking does not have access to fresh cross-user interaction data or newly trained backend models.

Traceability note. This record connects a stated project requirement to a concrete implementation or verification activity. It is retained as submission evidence so that the system can be assessed against observable behaviour rather than feature claims alone.

F.7 Local Tourism Assistance and Safety

Identifier	REQ-TRACE-07 / FR-11, FR-12, NFR-15
Purpose	Verify that local intelligence can answer supported tourism questions and prioritise emergency handling.
Preconditions	Offline knowledge assets and intelligence services are initialized; network connectivity is disabled.
Procedure or decision	Ask for a destination suggestion, a practical travel phrase, a follow-up question requiring context, and an emergency-style message. Inspect response mode and safety content.
Acceptance evidence	Supported tourism queries receive local retrieval or template responses; conversation state is retained; emergency content is handled by deterministic safety logic.
Residual risk	The assistant is not a substitute for emergency services, medical advice, or verified real-time trail conditions.

Traceability note. This record connects a stated project requirement to a concrete implementation or verification activity. It is retained as submission evidence so that the system can be assessed against observable behaviour rather than feature claims alone.

F.8 Backend Failure Degradation

Identifier	REQ-TRACE-08 / FR-20, NFR-08, NFR-13
Purpose	Confirm that a backend outage is visible but does not disable local application features.
Preconditions	The app is configured with a deliberately unreachable backend URL while local assets remain intact.
Procedure or decision	Launch the app, trigger a backend-enhanced action, inspect the user-facing state, then continue to discovery, map, saved items, local recommendations, and local chatbot functions.
Acceptance evidence	The application communicates that online services are unavailable and continues to expose its offline-capable workflows.
Residual risk	Remote-only capabilities such as authentication, fresh analytics, and online model responses remain unavailable until connectivity and backend health recover.

Traceability note. This record connects a stated project requirement to a concrete implementation or verification activity. It is retained as submission evidence so that the system can be assessed against observable behaviour rather than feature claims alone.

Appendix G

Data Dictionary and Dataset Evidence

The recommendation and discovery experience depends on consistent structured data. The following dictionary documents the principal destination fields used across Flutter and FastAPI components.

Table G.1: Destination data dictionary.

Field	Type	Meaning and use	Validation
id	string	Stable destination key used by UI, persistence, routes, interactions, and recommendation outputs.	Required; unique
name	string	Human-readable destination name displayed throughout the application.	Required; non-empty
description	string	Curated descriptive content for details, retrieval, and semantic embedding.	Required; non-empty
latitude	number	Geographic latitude used by maps, distance features, and routing.	-90 to 90
longitude	number	Geographic longitude used by maps, distance features, and routing.	-180 to 180
category	string	Primary tourism category such as village, lake, viewpoint, temple, or trek.	Controlled value
activities	list	Supported experiences such as trekking, culture, nature, photography, or pilgrimage.	Non-empty list

Field	Type	Meaning and use	Validation
best_season	list	Seasons in which the destination is most suitable.	Controlled values
budget_level	string	Relative cost category used by filters and contextual scoring.	Controlled value
accessibility	string	Relative access difficulty used in suitability filtering and explanations.	Controlled value
adventure_level	integer	Ordinal indicator of desired physical/adventure intensity.	Bounded scale
family_friendly	boolean	Indicates suitability for family-oriented discovery scenarios.	Boolean
cultural_richness	number	Relative cultural value used in filters and ranking features.	Bounded scale
nature_score	number	Relative natural-attraction value used in filters and ranking features.	Bounded scale
tags	list	Additional searchable and semantically useful descriptive labels.	Normalised strings
image	string	Local or resolvable image reference used by cards and detail presentation.	Asset existence
sbert_text	string	Prepared semantic text combining meaningful destination attributes.	Non-empty text
province	string	Administrative region used to constrain and describe coverage.	Gandaki scope

G.1 Dataset Identity and Uniqueness

Identifier	DATA-EVIDENCE-01
Purpose	Prevent ambiguous persistence, interaction, and recommendation references.
Preconditions	The complete destination asset is available to a validation script or model loader.
Procedure or decision	Collect every destination identifier and name; report empty values, duplicate identifiers, and suspicious duplicate normalized names; verify that referenced identifiers remain stable between app and backend assets.
Acceptance evidence	Every packaged destination has one stable unique identifier and a usable display name.
Residual risk	Name changes and future record merges require migration rules so saved items and interactions remain associated correctly.

Traceability note. This record connects a stated project requirement to a concrete implementation or verification activity. It is retained as submission evidence so that the system can be assessed against observable behaviour rather than feature claims alone.

G.2 Geographic Validity

Identifier	DATA-EVIDENCE-02
Purpose	Ensure that destination points can be rendered, measured, and routed without invalid coordinates.
Preconditions	All records have been parsed into numeric latitude and longitude values.
Procedure or decision	Check global coordinate bounds, inspect whether points fall within the intended Nepal/Gandaki coverage, and plot outliers for manual review.
Acceptance evidence	All intended destinations produce valid map points and no record silently defaults to an unrelated coordinate.
Residual risk	A valid coordinate does not guarantee a safe or publicly accessible arrival point; field verification remains necessary.

Traceability note. This record connects a stated project requirement to a concrete implementation or verification activity. It is retained as submission evidence so that the system can be assessed against observable behaviour rather than feature claims alone.

G.3 Semantic Retrieval Readiness

Identifier	DATA-EVIDENCE-03
Purpose	Confirm that recommendation and chatbot retrieval operate on informative text rather than destination names alone.
Preconditions	Each record exposes description, category, activity, tag, and prepared semantic text fields.
Procedure or decision	Inspect representative records across categories; compare source attributes with prepared semantic text; check for empty or duplicated semantic content; verify embedding-index alignment with identifiers.
Acceptance evidence	Semantic records contain enough distinct tourism context to support similarity search and scenario-oriented ranking.
Residual risk	Curated descriptions may reflect author bias and can underrepresent less documented communities.

Traceability note. This record connects a stated project requirement to a concrete implementation or verification activity. It is retained as submission evidence so that the system can be assessed against observable behaviour rather than feature claims alone.

G.4 Local Media Integrity

Identifier	DATA-EVIDENCE-04
Purpose	Verify that offline destination presentation can resolve packaged images reliably.
Preconditions	The Flutter asset manifest and local image directory are available.
Procedure or decision	Resolve each destination image reference, identify missing or duplicate assets, open a sample across image sizes, and inspect card/detail fallback behaviour.
Acceptance evidence	Intended local image references resolve and missing media does not crash a destination card or details screen.
Residual risk	Image licensing, accessibility text, and future content updates require an explicit governance process.

Traceability note. This record connects a stated project requirement to a concrete implementation or verification activity. It is retained as submission evidence so that the system can be assessed against observable behaviour rather than feature claims alone.

G.5 Accommodation Relationship Integrity

Identifier	DATA-EVIDENCE-05
Purpose	Confirm that accommodation records can be associated with their destination context.
Preconditions	The accommodation dataset and destination identifier set are available.
Procedure or decision	Validate accommodation identifiers, destination foreign-key references, coordinates, names, types, price data, and duplicated records; inspect destinations with zero and many linked accommodations.
Acceptance evidence	Each of the 771 intended accommodation records is parseable and every declared destination relationship resolves.
Residual risk	Availability and prices are not real-time and must be labelled accordingly in future booking-oriented releases.

Traceability note. This record connects a stated project requirement to a concrete implementation or verification activity. It is retained as submission evidence so that the system can be assessed against observable behaviour rather than feature claims alone.

Appendix H

Detailed API Contract Catalogue

Each contract sheet records the behaviour expected by the Flutter client and by administrative or evaluation tooling. Exact payloads remain versioned by the backend implementation; the sheets focus on externally observable obligations.

H.1 Health Check Contract

Identifier	API-01 / GET /health
Purpose	Provide a low-cost signal that the FastAPI process is reachable before online enhancement is attempted.
Preconditions	The backend process has started and its configured port is reachable.
Procedure or decision	Send an unauthenticated GET request; inspect status code and response body; repeat while a downstream optional model provider is unavailable.
Acceptance evidence	The endpoint returns a successful service-health response without requiring user credentials or an external model provider.
Residual risk	Process health alone does not prove that every optional dependency or data source is healthy.

Traceability note. This record connects a stated project requirement to a concrete implementation or verification activity. It is retained as submission evidence so that the system can be assessed against observable behaviour rather than feature claims alone.

H.2 Destination Collection Contract

Identifier	API-02 / GET /destinations
Purpose	Expose the destination catalogue for online consumers and consistency checks.
Preconditions	The backend destination repository has loaded its dataset.
Procedure or decision	Request the collection; inspect record count, identifiers, names, coordinates, and pagination or limit behaviour if configured.
Acceptance evidence	The response is valid JSON, uses stable identifiers, and exposes fields compatible with the client destination model.
Residual risk	Large future datasets require explicit pagination and version-aware client handling.

Traceability note. This record connects a stated project requirement to a concrete implementation or verification activity. It is retained as submission evidence so that the system can be assessed against observable behaviour rather than feature claims alone.

H.3 Destination Detail and Accommodation Contract

Identifier	API-03 / GET /destinations/{id} and accommodation route
Purpose	Expose one destination and its linked accommodation context.
Preconditions	A valid destination identifier and an invalid test identifier are available.
Procedure or decision	Request a valid detail record, request its accommodations, then request a missing identifier and inspect error semantics.
Acceptance evidence	Valid requests return the intended record and relationships; missing identifiers return a controlled not-found response rather than a server error.
Residual risk	Accommodation data remains informational unless integrated with a verified availability provider.

Traceability note. This record connects a stated project requirement to a concrete implementation or verification activity. It is retained as submission evidence so that the system can be assessed against observable behaviour rather than feature claims alone.

H.4 Recommendation Contract

Identifier	API-04 / POST /recommend
Purpose	Return ranked and explainable destination suggestions for a supplied user and context.
Preconditions	The destination index and recommendation service are initialized; a valid request payload is available.
Procedure or decision	Submit a cold-start request, a preference-rich request, and a request with contextual attributes; inspect ordering, scores, reasons, metadata, and repeatability.
Acceptance evidence	The endpoint returns a bounded ranked list with stable identifiers, interpretable reasons, and metadata sufficient for client presentation and analysis.
Residual risk	Sparse or synthetic interactions limit confidence in collaborative and learned ranking signals.

Traceability note. This record connects a stated project requirement to a concrete implementation or verification activity. It is retained as submission evidence so that the system can be assessed against observable behaviour rather than feature claims alone.

H.5 Similar Destination Contract

Identifier	API-05 / GET /similar/{id}
Purpose	Support related-destination discovery using semantic similarity.
Preconditions	A valid indexed destination identifier is available.
Procedure or decision	Request similar destinations, verify that the source destination is excluded, inspect similarity ordering, and test a missing identifier.
Acceptance evidence	The response contains valid alternatives with scores or ordering consistent with the semantic index.
Residual risk	Semantic similarity can reinforce catalogue-description bias and does not automatically provide diversity.

Traceability note. This record connects a stated project requirement to a concrete implementation or verification activity. It is retained as submission evidence so that the system can be assessed against observable behaviour rather than feature claims alone.

H.6 Interaction Logging Contract

Identifier	API-06 / POST /interactions and /interactions/batch
Purpose	Accept behavioural events used for analytics and future recommendation improvement.
Preconditions	The interaction repository is writable and test user/destination identifiers are available.
Procedure or decision	Submit a single event, submit a mixed batch, inspect accepted counts, test validation failures, and verify repository persistence.
Acceptance evidence	Valid events are stored once with required context; invalid events receive controlled validation responses; batch results expose partial-failure semantics.
Residual risk	Interaction collection requires consent, retention controls, and protection against event duplication or manipulation.

Traceability note. This record connects a stated project requirement to a concrete implementation or verification activity. It is retained as submission evidence so that the system can be assessed against observable behaviour rather than feature claims alone.

H.7 Chat Contract

Identifier	API-07 / POST /chat
Purpose	Provide online-enhanced tourism assistance while preserving controlled fallback behaviour.
Preconditions	The chat route is registered; at least one provider or fallback service is available.
Procedure or decision	Send representative discovery, route, translation, follow-up, unsupported, and provider-failure prompts; inspect response source and safety handling.
Acceptance evidence	The endpoint returns a structured response and does not expose provider exceptions directly to the client.
Residual risk	External provider content can be inaccurate; the client must preserve safety disclaimers and local fallback.

Traceability note. This record connects a stated project requirement to a concrete implementation or verification activity. It is retained as submission evidence so that the system can be assessed against observable behaviour rather than feature claims alone.

H.8 Offline Chat Contract

Identifier	API-08 / POST /chat/offline
Purpose	Expose the backend's deterministic offline-oriented retrieval response for explicit testing and fallback use.
Preconditions	Offline chat data and service initialization have completed.
Procedure or decision	Submit supported and unsupported tourism prompts and inspect intent, response text, and destination references.
Acceptance evidence	The endpoint responds without calling a cloud language model and returns controlled content for supported local knowledge.
Residual risk	The available response range is narrower than generative online assistance.

Traceability note. This record connects a stated project requirement to a concrete implementation or verification activity. It is retained as submission evidence so that the system can be assessed against observable behaviour rather than feature claims alone.

H.9 Translation Contract

Identifier	API-09 / POST / <code>translate</code>
Purpose	Provide supported travel translation with explicit language-pair and failure semantics.
Preconditions	Translation resources are initialized and a supported-pair matrix is documented.
Procedure or decision	Submit representative supported phrases, mixed-case input, romanised Nepali, an unsupported pair, and empty or excessive input.
Acceptance evidence	Supported requests return structured translated text and language metadata; invalid requests receive controlled validation errors.
Residual risk	Coverage remains domain-specific and should not be represented as general professional translation.

Traceability note. This record connects a stated project requirement to a concrete implementation or verification activity. It is retained as submission evidence so that the system can be assessed against observable behaviour rather than feature claims alone.

H.10 Authentication Contract

Identifier	API-10 / registration, login, and current-user routes
Purpose	Provide identity for optional synchronized and personalized online capability.
Preconditions	The authentication repository and secret configuration are available.
Procedure or decision	Register a new account, reject duplicate or invalid input, login with correct and incorrect credentials, then access the current-user route with valid, expired, and missing tokens.
Acceptance evidence	Passwords are not returned or stored in plain text; successful login returns a usable token; protected access rejects invalid credentials.
Residual risk	Authentication security also depends on deployment TLS, secret rotation, rate limiting, and production monitoring.

Traceability note. This record connects a stated project requirement to a concrete implementation or verification activity. It is retained as submission evidence so that the system can be assessed against observable behaviour rather than feature claims alone.

Appendix I

Verification Test Catalogue

The following cases describe repeatable acceptance and regression checks. They supplement automated unit and widget tests with integrated offline-first scenarios that cross storage, presentation, networking, and intelligence boundaries.

I.1 Cold Offline Startup Test

Identifier	TEST-ACC-01
Purpose	Test the most important failure condition: no connection at the start of a session.
Preconditions	Install the release build; clear recent tasks; enable airplane mode before launching.
Procedure or decision	Launch, complete or skip onboarding, open home, search, details, map, saved items, recommendations, chatbot, and translation in sequence.
Acceptance evidence	Core local screens remain reachable; no infinite loading state or uncaught network exception prevents continued use.
Residual risk	Some screens can correctly show that optional online enhancement is unavailable.

Traceability note. This record connects a stated project requirement to a concrete implementation or verification activity. It is retained as submission evidence so that the system can be assessed against observable behaviour rather than feature claims alone.

I.2 Warm Online-to-Offline Transition Test

Identifier	TEST-ACC-02
Purpose	Check graceful degradation when connectivity disappears during active use.
Preconditions	The application is open and the backend is initially reachable.
Procedure or decision	Load online recommendations and a road route, disable connectivity, repeat both actions, then open local content and send a chatbot query.
Acceptance evidence	Cached or local alternatives are used where designed; the state change is communicated without a crash.
Residual risk	An in-flight remote request may take until its configured timeout before fallback occurs.

Traceability note. This record connects a stated project requirement to a concrete implementation or verification activity. It is retained as submission evidence so that the system can be assessed against observable behaviour rather than feature claims alone.

I.3 Offline-to-Online Recovery Test

Identifier	TEST-ACC-03
Purpose	Check that the app recovers online capability without requiring reinstallation or data loss.
Preconditions	The app has been used offline and contains queued interactions.
Procedure or decision	Restore connectivity, trigger health checking or retry, request an online route, and observe interaction synchronization.
Acceptance evidence	Online capability returns; queued events can be submitted; local saved state and caches remain intact.
Residual risk	Recovery timing depends on platform connectivity signals and backend availability.

Traceability note. This record connects a stated project requirement to a concrete implementation or verification activity. It is retained as submission evidence so that the system can be assessed against observable behaviour rather than feature claims alone.

I.4 Route Cache Precedence Test

Identifier	TEST-ACC-04
Purpose	Prove that an eligible cached route takes precedence and avoids unnecessary network use.
Preconditions	A known OSRM route has been cached and remains within the configured freshness period.
Procedure or decision	Request the identical coordinate pair with network available and inspect route source and outgoing traffic; repeat after expiry or cache removal.
Acceptance evidence	The fresh cache serves the repeat request; expired or absent entries allow later routing tiers to execute.
Residual risk	Coordinate rounding and cache-key design can affect hit rate for nearly identical points.

Traceability note. This record connects a stated project requirement to a concrete implementation or verification activity. It is retained as submission evidence so that the system can be assessed against observable behaviour rather than feature claims alone.

I.5 Haversine Fallback Test

Identifier	TEST-ACC-05
Purpose	Confirm that a disconnected cache miss still returns a useful directional estimate.
Preconditions	No relevant route is cached and the device is offline.
Procedure or decision	Request a route between two valid destinations and compare reported straight-line distance with an independently calculated Haversine value.
Acceptance evidence	A clearly identified fallback route is returned with plausible distance and without turn-by-turn claims.
Residual risk	Terrain, roads, bridges, closures, and walking feasibility are not represented by the fallback line.

Traceability note. This record connects a stated project requirement to a concrete implementation or verification activity. It is retained as submission evidence so that the system can be assessed against observable behaviour rather than feature claims alone.

I.6 Recommendation Scenario Evaluation Test

Identifier	TEST-REC-01
Purpose	Measure ranked retrieval quality against the report's five defined tourism scenarios.
Preconditions	Scenario queries, relevance sets, and the candidate destination catalogue are fixed for the run.
Procedure or decision	Generate top-ten rankings for every scenario and calculate Precision@10, Recall@10, nDCG@10, and MRR using the same relevance judgments.
Acceptance evidence	Metric calculation is reproducible and results can be compared with the documented hybrid recommender values.
Residual risk	Five scenarios are useful for project evaluation but insufficient to establish population-level effectiveness.

Traceability note. This record connects a stated project requirement to a concrete implementation or verification activity. It is retained as submission evidence so that the system can be assessed against observable behaviour rather than feature claims alone.

I.7 Recommendation Explanation Test

Identifier	TEST-REC-02
Purpose	Verify that recommendation reasons correspond to actual ranking features and destination attributes.
Preconditions	A set of recommendations with score components and destination records is available.
Procedure or decision	Inspect reasons for season, activity, budget, distance, culture, nature, popularity, and similar-item signals; compare each statement with source attributes.
Acceptance evidence	Reasons are understandable and do not claim attributes that the destination record or ranking path does not support.
Residual risk	Simplified explanations cannot fully describe every interaction within a learned ranking model.

Traceability note. This record connects a stated project requirement to a concrete implementation or verification activity. It is retained as submission evidence so that the system can be assessed against observable behaviour rather than feature claims alone.

I.8 Cold-Start Recommendation Test

Identifier	TEST-REC-03
Purpose	Confirm that a new user receives useful, diverse suggestions without an interaction history.
Preconditions	Use a new anonymous identifier or clear local recommendation history.
Procedure or decision	Request recommendations with no preferences, then with onboarding preferences; inspect relevance, diversity, and explanation changes.
Acceptance evidence	The system supplies reasonable defaults and responds to explicit preferences without requiring collaborative history.
Residual risk	Popularity-based defaults can overexpose already prominent destinations.

Traceability note. This record connects a stated project requirement to a concrete implementation or verification activity. It is retained as submission evidence so that the system can be assessed against observable behaviour rather than feature claims alone.

I.9 Emergency Detection Test

Identifier	TEST-AI-01
Purpose	Verify deterministic priority handling of messages that indicate immediate danger or need for assistance.
Preconditions	The local intelligence pipeline is initialized and external AI providers are disabled.
Procedure or decision	Submit representative emergency, injury, lost-person, and benign messages in supported language forms; inspect detection and response precedence.
Acceptance evidence	Emergency-like messages trigger the safety layer before normal tourism recommendation generation; benign queries avoid false emergency escalation.
Residual risk	Language variation and ambiguous messages require continuing review with domain experts.

Traceability note. This record connects a stated project requirement to a concrete implementation or verification activity. It is retained as submission evidence so that the system can be assessed against observable behaviour rather than feature claims alone.

I.10 Dialogue Continuity Test

Identifier	TEST-AI-02
Purpose	Check that follow-up questions can use recent conversation context.
Preconditions	Start a fresh conversation and ask for a destination recommendation with explicit constraints.
Procedure or decision	Send follow-ups that omit the destination name but refer to cost, route, season, and alternatives; then reset the conversation.
Acceptance evidence	Relevant recent slots and entities are retained before reset and are not incorrectly reused after reset.
Residual risk	Long conversations and topic changes can still create ambiguity requiring clarification.

Traceability note. This record connects a stated project requirement to a concrete implementation or verification activity. It is retained as submission evidence so that the system can be assessed against observable behaviour rather than feature claims alone.

I.11 Translation Coverage Test

Identifier	TEST-AI-03
Purpose	Evaluate supported travel phrases, normalization, and controlled fallback behaviour.
Preconditions	The local glossary, phrasebook, templates, and available online adapter are known.
Procedure or decision	Test exact phrases, punctuation and case variants, romanised Nepali, Devanagari input, unsupported text, and provider unavailability.
Acceptance evidence	Supported local phrases translate consistently; response metadata distinguishes translation source or fallback state.
Residual risk	General prose, specialist terminology, and dialect variation remain outside validated coverage.

Traceability note. This record connects a stated project requirement to a concrete implementation or verification activity. It is retained as submission evidence so that the system can be assessed against observable behaviour rather than feature claims alone.

I.12 Static Analysis and Automated Regression Test

Identifier	TEST-ENG-01
Purpose	Provide repeatable engineering-quality evidence for Flutter and backend components.
Preconditions	Dependencies are installed in the documented environments.
Procedure or decision	Run Flutter static analysis, focused intelligence and recommendation tests, backend pytest, and relevant widget tests; retain command, date, environment, and outcome.
Acceptance evidence	Static analysis and focused suites produce a clear pass/fail signal; known failures are documented rather than concealed.
Residual risk	Environment-specific timing and golden-image rendering can differ across machines and SDK versions.

Traceability note. This record connects a stated project requirement to a concrete implementation or verification activity. It is retained as submission evidence so that the system can be assessed against observable behaviour rather than feature claims alone.

Appendix J

User Manual and Operational Workflows

This manual describes the intended end-user workflows. Screen names may evolve, but the operational sequence and offline expectations remain part of the project's acceptance baseline.

J.1 First Launch and Onboarding

Identifier	USER-01
Purpose	Help a first-time traveler configure useful preferences without blocking immediate exploration.
Preconditions	The application is installed and launched for the first time.
Procedure or decision	Review onboarding, choose available preferences where offered, grant only required permissions, and continue to the main dashboard.
Acceptance evidence	The user reaches the application shell; declared preferences can influence discovery and recommendation; denied optional permissions do not prevent catalogue browsing.
Residual risk	Location-dependent convenience is reduced when location permission is denied.

Traceability note. This record connects a stated project requirement to a concrete implementation or verification activity. It is retained as submission evidence so that the system can be assessed against observable behaviour rather than feature claims alone.

J.2 Browse and Search Destinations

Identifier	USER-02
Purpose	Find relevant destinations from the bundled Gandaki catalogue.
Preconditions	The home or discovery screen is open.
Procedure or decision	Scroll destination content, enter a search term, apply available filters, clear the query, and open a result.
Acceptance evidence	Results respond to the query and filters; destination cards remain readable; clearing controls restores broader discovery.
Residual risk	Search quality depends on the vocabulary represented in the curated dataset.

Traceability note. This record connects a stated project requirement to a concrete implementation or verification activity. It is retained as submission evidence so that the system can be assessed against observable behaviour rather than feature claims alone.

J.3 Read Destination Details

Identifier	USER-03
Purpose	Inspect practical and descriptive information before deciding to travel.
Preconditions	A destination card or recommendation has been selected.
Procedure or decision	Review description, images, category, activities, season, budget, accessibility, location, and available accommodation context; follow map or save actions.
Acceptance evidence	Information is presented consistently and available local assets remain visible without a connection.
Residual risk	Static descriptions cannot replace local safety advice or real-time access information.

Traceability note. This record connects a stated project requirement to a concrete implementation or verification activity. It is retained as submission evidence so that the system can be assessed against observable behaviour rather than feature claims alone.

J.4 Save and Review a Destination

Identifier	USER-04
Purpose	Retain personal travel interests and feedback locally.
Preconditions	A destination detail view is open and local storage is available.
Procedure or decision	Toggle the saved state, locate the destination in saved content, create or update a review, restart the app, and inspect persistence.
Acceptance evidence	Saved and review state remains available after restart and can be intentionally changed or removed.
Residual risk	Local-only state is lost if application storage is cleared or the app is uninstalled.

Traceability note. This record connects a stated project requirement to a concrete implementation or verification activity. It is retained as submission evidence so that the system can be assessed against observable behaviour rather than feature claims alone.

J.5 Explore the Offline Map

Identifier	USER-05
Purpose	Use geographic context in places with unreliable mobile data.
Preconditions	The map screen is available and the bundled tile database is packaged.
Procedure or decision	Open the map, move to a known Gandaki area, pan and zoom within supported coverage, select a destination marker, and inspect the details action.
Acceptance evidence	The map remains usable within packaged coverage and destination points connect to discovery workflows.
Residual risk	The bundled raster map is a snapshot and does not show live closures or every street-level detail.

Traceability note. This record connects a stated project requirement to a concrete implementation or verification activity. It is retained as submission evidence so that the system can be assessed against observable behaviour rather than feature claims alone.

J.6 Request Directions

Identifier	USER-06
Purpose	Obtain the best available route representation for the current connectivity state.
Preconditions	Origin and destination coordinates are available; location permission is granted if current position is used.
Procedure or decision	Request a route; inspect whether it is cached, road-aware online, or straight-line fallback; review distance and any warning; repeat offline.
Acceptance evidence	The route source is understandable and the app does not imply road guidance when only a Haversine estimate is available.
Residual risk	Users must verify local conditions and should not rely solely on the fallback for safety-critical navigation.

Traceability note. This record connects a stated project requirement to a concrete implementation or verification activity. It is retained as submission evidence so that the system can be assessed against observable behaviour rather than feature claims alone.

J.7 Receive Personalized Recommendations

Identifier	USER-07
Purpose	Discover destinations that align with interests, season, budget, accessibility, and context.
Preconditions	The recommendation view is open and local destination data has loaded.
Procedure or decision	Provide or change available preference controls, request recommendations, read explanation text, open a suggested destination, and repeat without connectivity.
Acceptance evidence	Ranked results update meaningfully and expose reasons that help the user judge suitability.
Residual risk	Recommendations assist discovery but do not guarantee satisfaction, safety, or availability.

Traceability note. This record connects a stated project requirement to a concrete implementation or verification activity. It is retained as submission evidence so that the system can be assessed against observable behaviour rather than feature claims alone.

J.8 Use the Tourism Chatbot

Identifier	USER-08
Purpose	Ask natural-language tourism questions and receive the best available local or online-enhanced response.
Preconditions	The chatbot screen is open.
Procedure or decision	Ask a specific question, use a follow-up, request an alternative, disconnect the network, and ask another supported local question.
Acceptance evidence	The conversation remains usable; response state distinguishes fallback where appropriate; local questions continue to receive controlled assistance.
Residual risk	Generated or retrieved answers can be incomplete and should be verified for high-stakes decisions.

Traceability note. This record connects a stated project requirement to a concrete implementation or verification activity. It is retained as submission evidence so that the system can be assessed against observable behaviour rather than feature claims alone.

J.9 Translate a Travel Phrase

Identifier	USER-09
Purpose	Support practical communication using available English and Nepali resources.
Preconditions	The translation screen is open and source/target languages can be selected.
Procedure or decision	Enter a supported travel phrase, translate it, swap direction where supported, test romanised or Devanagari input, and repeat offline.
Acceptance evidence	Supported content produces a readable result and offline resources remain available for their declared coverage.
Residual risk	The feature is not a certified interpreter and should not be relied on for legal or medical communication.

Traceability note. This record connects a stated project requirement to a concrete implementation or verification activity. It is retained as submission evidence so that the system can be assessed against observable behaviour rather than feature claims alone.

J.10 Recognise Offline and Recovery States

Identifier	USER-10
Purpose	Help users understand which capabilities remain local and which require the network.
Preconditions	The application is running and connectivity can be toggled.
Procedure or decision	Disable the network, observe messaging while using local features, attempt an online enhancement, restore connectivity, and retry.
Acceptance evidence	The application communicates backend or connectivity loss without hiding local capability and recovers online actions when service returns.
Residual risk	Connectivity indicators can lag behind the actual reachability of a particular external service.

Traceability note. This record connects a stated project requirement to a concrete implementation or verification activity. It is retained as submission evidence so that the system can be assessed against observable behaviour rather than feature claims alone.

Appendix K

Deployment, Configuration, and Troubleshooting

K.1 Flutter Development Setup

Identifier	OPS-01
Purpose	Provide a repeatable setup path for application development and verification.
Preconditions	Flutter, Dart, Android tooling, and a supported device or emulator are available.
Procedure or decision	Open the app directory, obtain dependencies, inspect environment configuration, run static analysis and focused tests, then launch a debug build.
Acceptance evidence	Dependencies resolve, analysis completes, tests provide a clear result, and the app launches against the intended local or remote backend configuration.
Residual risk	SDK and plugin upgrades can change build behaviour; versions should be recorded for reproducibility.

Traceability note. This record connects a stated project requirement to a concrete implementation or verification activity. It is retained as submission evidence so that the system can be assessed against observable behaviour rather than feature claims alone.

K.2 Backend Development Setup

Identifier	OPS-02
Purpose	Provide a repeatable FastAPI setup for recommendation, chat, authentication, and analytics development.
Preconditions	A supported Python environment and project dependencies are available.
Procedure or decision	Configure environment variables, initialize required data, run backend tests, start the API server, and request the health endpoint.
Acceptance evidence	Tests and health checking provide a clear operational signal and registered routes are visible through FastAPI documentation.
Residual risk	Optional provider keys and production databases require separate secure configuration.

Traceability note. This record connects a stated project requirement to a concrete implementation or verification activity. It is retained as submission evidence so that the system can be assessed against observable behaviour rather than feature claims alone.

K.3 Backend URL and Environment Switching

Identifier	OPS-03
Purpose	Avoid scattered host literals and support emulator, device, and deployed environments.
Preconditions	The central application backend configuration is available.
Procedure or decision	Set the intended backend environment or base URL, launch the app, inspect the health request, and verify that feature services derive their URIs from the central configuration.
Acceptance evidence	Runtime requests use the selected central base URL and backend failure is surfaced through the intended offline UX.
Residual risk	Physical devices, emulators, and remote deployments have different network addressing and firewall constraints.

Traceability note. This record connects a stated project requirement to a concrete implementation or verification activity. It is retained as submission evidence so that the system can be assessed against observable behaviour rather than feature claims alone.

K.4 Android Release Build and Asset Verification

Identifier	OPS-04
Purpose	Produce a distributable package containing the offline assets required by the project.
Preconditions	Release signing and Flutter Android tooling are configured.
Procedure or decision	Build the release APK, inspect size, install it on a clean device, enable airplane mode, and verify destination data, images, MBTiles, and local intelligence assets.
Acceptance evidence	The release installs and core offline workflows operate without relying on development-machine files.
Residual risk	Bundled assets increase download size and must be revalidated after asset-manifest changes.

Traceability note. This record connects a stated project requirement to a concrete implementation or verification activity. It is retained as submission evidence so that the system can be assessed against observable behaviour rather than feature claims alone.

K.5 Operational Troubleshooting Guide

Identifier	OPS-05
Purpose	Provide a systematic response to common failures without masking data loss or unsafe behaviour.
Preconditions	Logs, app settings, and health endpoints are available to the developer or tester.
Procedure or decision	For startup failure, inspect asset parsing and storage initialization; for backend failure, inspect base URL and health; for blank maps, inspect MBTiles packaging; for stale routes, inspect cache age; for AI failure, inspect local initialization before provider status.
Acceptance evidence	Troubleshooting isolates the affected tier and preserves local functionality wherever possible.
Residual risk	Clearing application data is a last resort because it removes saved local state and can hide the original fault.

Traceability note. This record connects a stated project requirement to a concrete implementation or verification activity. It is retained as submission evidence so that the system can be assessed against observable behaviour rather than feature claims alone.

Appendix L

Architecture Decision Records

L.1 Use Flutter for the Mobile Client

Identifier	ADR-01 / Accepted
Purpose	Deliver a maintainable Android application with a consistent component model and future cross-platform potential.
Preconditions	The project requires a rich mobile interface, local storage, maps, and broad plugin support.
Procedure or decision	Adopt Flutter and Dart; structure features around presentation, domain models, and services; use platform integrations only where required.
Acceptance evidence	The implemented client provides the intended screens, local data access, maps, and testable Dart services from one codebase.
Residual risk	Plugin quality and native build complexity can affect platform-specific behaviour.

Traceability note. This record connects a stated project requirement to a concrete implementation or verification activity. It is retained as submission evidence so that the system can be assessed against observable behaviour rather than feature claims alone.

L.2 Treat Offline Operation as the Default Architecture

Identifier	ADR-02 / Accepted
Purpose	Preserve useful travel support in the low-connectivity conditions central to the research problem.
Preconditions	Core catalogue, media, maps, routing fallback, storage, recommendations, and assistance can be packaged or computed locally.
Procedure or decision	Design local capability first and layer remote enhancement behind reachability checks, timeouts, caching, queues, and controlled fallback.
Acceptance evidence	The application remains useful during backend or network loss and does not place every workflow behind authentication.
Residual risk	Offline assets are static and increase package size; synchronization requires careful conflict and freshness handling.

Traceability note. This record connects a stated project requirement to a concrete implementation or verification activity. It is retained as submission evidence so that the system can be assessed against observable behaviour rather than feature claims alone.

L.3 Use FastAPI for Online Enhancement Services

Identifier	ADR-03 / Accepted
Purpose	Expose typed Python services for recommendation, semantic search, evaluation, chat, analytics, and authentication.
Preconditions	The recommendation and NLP ecosystem is primarily Python-based and benefits from automatic API documentation.
Procedure or decision	Implement versioned route modules and application services in FastAPI; keep offline-critical capability in Flutter.
Acceptance evidence	Online services can evolve independently while the client preserves local fallback.
Residual risk	A production deployment still requires TLS, observability, scaling, rate limiting, and secret management.

Traceability note. This record connects a stated project requirement to a concrete implementation or verification activity. It is retained as submission evidence so that the system can be assessed against observable behaviour rather than feature claims alone.

L.4 Combine SQLite, Hive, and SharedPreferences

Identifier	ADR-04 / Accepted
Purpose	Match local persistence mechanisms to relational records, high-frequency cached objects, queues, and lightweight settings.
Preconditions	Saved destinations, reviews, routes, recommendation caches, interactions, and preferences have different access patterns.
Procedure or decision	Use SQLite for relational saved/review/route data, Hive for recommendation cache and pending interactions, and SharedPreferences for small settings.
Acceptance evidence	Each local data category has a storage mechanism appropriate to its structure and lifecycle.
Residual risk	Multiple stores increase migration, backup, and debugging complexity.

Traceability note. This record connects a stated project requirement to a concrete implementation or verification activity. It is retained as submission evidence so that the system can be assessed against observable behaviour rather than feature claims alone.

L.5 Bundle MBTiles for Offline Map Coverage

Identifier	ADR-05 / Accepted
Purpose	Ensure that geographic context remains available without run-time tile downloads.
Preconditions	A bounded Gandaki coverage area and practical zoom range are acceptable for the project.
Procedure or decision	Package a local raster MBTiles database and use online services only as optional enhancement.
Acceptance evidence	The app renders the intended Gandaki map region in airplane mode.
Residual risk	Higher zoom levels and expanded coverage would substantially increase package size.

Traceability note. This record connects a stated project requirement to a concrete implementation or verification activity. It is retained as submission evidence so that the system can be assessed against observable behaviour rather than feature claims alone.

L.6 Use Cache, OSRM, then Haversine for Routing

Identifier	ADR-06 / Accepted
Purpose	Provide the best available route response while guaranteeing a disconnected fallback.
Preconditions	Road-aware offline routing data is too large or complex for the current scope.
Procedure or decision	Resolve routes through a fresh SQLite cache, then online OSRM, then a clearly identified straight-line Haversine approximation.
Acceptance evidence	Repeated routes are efficient, online routes can be road-aware, and disconnected cache misses still return useful orientation.
Residual risk	The fallback must never be confused with safe road or trail instructions.

Traceability note. This record connects a stated project requirement to a concrete implementation or verification activity. It is retained as submission evidence so that the system can be assessed against observable behaviour rather than feature claims alone.

L.7 Hybrid Explainable Recommendation Pipeline

Identifier	ADR-07 / Accepted
Purpose	Balance semantic relevance, context, interaction evidence, diversity, and understandable user-facing reasons.
Preconditions	The catalogue contains rich text and attributes, while real-user interaction volume remains limited.
Procedure or decision	Combine semantic candidates, collaborative/popularity signals, contextual reranking, optional learned rescoring, diversity control, and explanation generation.
Acceptance evidence	Scenario evaluation demonstrates strong first-result placement and exposes measurable recall limitations.
Residual risk	Learned and collaborative components require representative consented usage data before strong production claims.

Traceability note. This record connects a stated project requirement to a concrete implementation or verification activity. It is retained as submission evidence so that the system can be assessed against observable behaviour rather than feature claims alone.

L.8 Keep Deterministic Local Safety and Intelligence

Identifier	ADR-08 / Accepted
Purpose	Avoid making tourism assistance and emergency handling wholly dependent on a network or generative provider.
Preconditions	Local language processing, retrieval, dialogue state, phrase resources, and emergency rules can run on-device.
Procedure or decision	Run deterministic safety handling and local intelligence for every relevant message, using online language models only as an optional enhancement.
Acceptance evidence	Supported queries and emergency-like messages retain controlled handling during provider or network failure.
Residual risk	Local coverage is finite and requires continued multilingual testing and expert review.

Traceability note. This record connects a stated project requirement to a concrete implementation or verification activity. It is retained as submission evidence so that the system can be assessed against observable behaviour rather than feature claims alone.

Appendix M

Selected Implementation Listings

The following listings are drawn directly from the implemented project. They are included as technical evidence rather than as a claim that every source file belongs in the dissertation. Line numbers support discussion and review.

M.1 FastAPI Recommendation Route

Listing M.1: Implemented recommendation API route.

```

1 import logging
2 from functools import lru_cache
3 from uuid import uuid4
4
5 from fastapi import APIRouter, Header, Response
6
7 from backend.api.v1.auth_dependencies import optional_authenticated_user_id
8 from backend.application.dto.requests import RecommendationRequestDto
9 from backend.application.dto.responses import RecommendationResponseDto
10 from backend.application.services.interaction_logging_service import (
11     InteractionLoggingService,
12 )
13 from backend.cache import get_cache, make_cache_key, set_cache
14 from backend.core.config import settings
15
16 router = APIRouter()
17 logger = logging.getLogger(__name__)
18
19
20 @lru_cache(maxsize=1)
21 def _get_service():
22     from backend.application.services.recommendation_service import (
23         RecommendationService,
24     )
25
26     return RecommendationService()
27
28
29 @lru_cache(maxsize=1)
30 def _get_interaction_logger() -> InteractionLoggingService:
31     return InteractionLoggingService()
32
33
34 @router.post("", response_model=RecommendationResponseDto)
35 def recommend(
36     payload: RecommendationRequestDto,
37     response: Response,
38     authorization: str | None = Header(default=None),
39 ):
40     """
41     Main recommendation endpoint.
42     Send user preferences and receive ranked destinations with scores and reasons.
43     """
44     authenticated_user_id = optional_authenticated_user_id(authorization)
45     if authenticated_user_id is not None:
46         payload = payload.model_copy(update={"user_id": authenticated_user_id})
47
48     preferences = _cache_preferences(payload)
49     cache_key = make_cache_key(payload.user_id, preferences)
50     cached = get_cache(cache_key)
51     recommendation_id = str(uuid4())
52     user_id = payload.user_id or "anonymous"
53
54     if cached is not None:
55         response.headers["X-Cache"] = "HIT"

```

```
56         result = RecommendationResponseDto.model_validate(cached).model_copy(
57             update={
58                 "recommendation_id": recommendation_id,
59                 "pipeline_used": "cached",
60             }
61         )
62         _log_recommendation_response(
63             user_id=user_id,
64             recommendation_id=recommendation_id,
65             result=result,
66             pipeline_used="cached",
67         )
68         response.headers["X-Recommendation-ID"] = recommendation_id
69         return result
70
71     result = _get_service().recommend(payload)
72     set_cache(
73         cache_key,
74         result.model_dump(mode="json"),
75         settings.redis_cache_ttl_seconds,
76     )
77     pipeline_used = "offline" if settings.offline_mode else "online"
78     result = result.model_copy(
79         update={
80             "recommendation_id": recommendation_id,
81             "pipeline_used": pipeline_used,
82         }
83     )
84     _log_recommendation_response(
85         user_id=user_id,
86         recommendation_id=recommendation_id,
87         result=result,
88         pipeline_used=pipeline_used,
89     )
90     response.headers["X-Cache"] = "MISS"
91     response.headers["X-Recommendation-ID"] = recommendation_id
92     return result
93
94
95 def _cache_preferences(payload: RecommendationRequestDto) -> dict:
96     return {
97         "activity": payload.activity,
98         "budget": payload.budget,
99         "season": payload.season,
100         "vibe": payload.vibe,
101         "family_friendly": payload.family_friendly,
102         "adventure_level": payload.adventure_level,
103         "seed_destination_id": payload.seed_destination_id,
104         "top_k": payload.top_k,
105         "semantic_only": payload.semantic_only,
106     }
107
108
109 def _log_recommendation_response(
110     *,
111     user_id: str,
112     recommendation_id: str,
113     result: RecommendationResponseDto,
114     pipeline_used: str,
```

```
115 ) -> None:
116     try:
117         _get_interaction_logger().log_recommendation_response(
118             user_id=user_id,
119             recommendation_id=recommendation_id,
120             recommended_destination_ids=[item.id for item in result.results],
121             pipeline_used=pipeline_used,
122         )
123     except Exception as exc:
124         logger.warning(
125             "Failed to log recommendation response %s: %s",
126             recommendation_id,
127             exc,
128         )
```

M.2 Backend Recommendation Application Service

Listing M.2: Implemented hybrid recommendation application service.

```
1  """
2  Recommendation pipeline with ML ranker integration.
3
4  Pipeline:
5      1. Build preference query
6      2. Retrieve candidates using SBERT
7      3. Add collaborative/popularity scores
8      4. Contextual reranking
9      5. ML ranker rescoring
10     6. Explainable recommendation output
11  """
12
13  from __future__ import annotations
14
15  from backend.application.dto.requests import RecommendationRequestDto
16  from backend.application.dto.responses import (
17      RecommendationResponseDto,
18      RecommendationResponseItemDto,
19  )
20  from backend.core.config import settings
21  from backend.infrastructure.explain.recommendation_explainer import RecommendationExplainer
22  from backend.infrastructure.ml.candidate_retriever import CandidateRetriever
23  from backend.infrastructure.ml.collaborative_filter import CollaborativeFilter
24  from backend.infrastructure.ml.contextual_reranker import ContextualReranker
25  from backend.infrastructure.ml.ranker import build_ranker_features, get_ranking_model
26  from backend.infrastructure.ml.sbert_encoder import PreferenceQueryBuilder
27  from backend.infrastructure.repositories.json_accommodation_repository import (
28      JsonAccommodationRepository,
29  )
30  from backend.infrastructure.repositories.json_destination_repository import (
31      JsonDestinationRepository,
32  )
33  from backend.infrastructure.repositories.interaction_repository_factory import (
34      build_interaction_repository,
35  )
36
37
38  _COLD_START_POPULARITY_WEIGHT = 0.15
39  _COLD_START_COLLABORATIVE_WEIGHT = 0.0
40
41  _WARM_POPULARITY_WEIGHT = 0.15
42  _WARM_COLLABORATIVE_WEIGHT = 0.85
43
44
45  class RecommendationService:
46      """
47      Recommendation pipeline:
48      SBERT retrieval -> collaborative/contextual reranking -> ML ranker -> explanation.
49      """
50
51      def __init__(self) -> None:
52          destination_repo = JsonDestinationRepository()
53          accommodation_repo = JsonAccommodationRepository()
54
55          self._interaction_repo = build_interaction_repository()
```

```
56
57     self._destinations = destination_repo.get_all()
58     self._accommodations = accommodation_repo.get_all()
59     self._destination_by_id = {d.id: d for d in self._destinations}
60
61     self._query_builder = PreferenceQueryBuilder()
62     self._retriever = CandidateRetriever(self._destinations)
63     self._reranker = ContextualReranker()
64     self._explainer = RecommendationExplainer()
65
66     self._ranker = get_ranking_model()
67     self._ranker.ensure_loaded()
68
69 def recommend(self, request: RecommendationRequestDto) -> RecommendationResponseDto:
70     query_text = self._query_builder.build(
71         activity=request.activity,
72         budget=request.budget,
73         season=request.season,
74         vibe=request.vibe,
75         family_friendly=request.family_friendly,
76         adventure_level=request.adventure_level,
77     )
78
79     candidates = self._retriever.retrieve(
80         query_text,
81         top_k=settings.retrieve_top_k,
82         activity=request.activity,
83         vibe=request.vibe,
84         season=request.season,
85         budget=request.budget,
86     )
87
88     if request.semantic_only:
89         return self._build_semantic_only_response(candidates, request)
90
91     candidate_ids = [c["destination"].id for c in candidates]
92
93     interactions = self._interaction_repo.get_all()
94     collab_filter = CollaborativeFilter(interactions)
95
96     raw_collab = collab_filter.score_candidates(
97         user_id=request.user_id or "",
98         candidate_ids=candidate_ids,
99     )
100
101     popularity = collab_filter.popular_destinations(candidate_ids)
102     is_cold_start = self._is_cold_start(raw_collab)
103
104     if is_cold_start:
105         blended_scores = {
106             cid: (
107                 _COLD_START_COLLABORATIVE_WEIGHT * raw_collab.get(cid, 0.0)
108                 + _COLD_START_POPULARITY_WEIGHT * popularity.get(cid, 0.0)
109             )
110             for cid in candidate_ids
111         }
112     else:
113         blended_scores = {
114             cid: (
```

```
115         _WARM_COLLABORATIVE_WEIGHT * raw_collab.get(cid, 0.0)
116         + _WARM_POPULARITY_WEIGHT * popularity.get(cid, 0.0)
117     )
118     for cid in candidate_ids
119 }
120
121 contextual_pool_k = min(
122     len(candidates),
123     max(request.top_k * 3, request.top_k),
124 )
125
126 ranked = self._reranker.rerank(
127     candidates=candidates,
128     accommodations=self._accommodations,
129     collaborative_scores=blended_scores,
130     activity=request.activity,
131     budget=request.budget,
132     season=request.season,
133     vibe=request.vibe,
134     family_friendly=request.family_friendly,
135     adventure_level=request.adventure_level,
136     top_k=contextual_pool_k,
137 )
138
139 ranked = self._apply_ml_ranking(
140     ranked=ranked,
141     request=request,
142     top_k=request.top_k,
143 )
144
145 items = []
146
147 for recommendation in ranked:
148     destination = self._destination_by_id[recommendation.id]
149
150     recommendation.reasons = self._explainer.build(
151         recommendation=recommendation,
152         destination=destination,
153         activity=request.activity,
154         budget=request.budget,
155         season=request.season,
156         vibe=request.vibe,
157         family_friendly=request.family_friendly,
158         is_cold_start=is_cold_start,
159     )
160
161     recommendation.metadata["is_cold_start"] = str(is_cold_start).lower()
162     recommendation.metadata["popularity_score"] = (
163         f"{popularity.get(recommendation.id, 0.0):.4f}"
164     )
165
166     items.append(self._to_dto(recommendation))
167
168 return RecommendationResponseDto(results=items, total=len(items))
169
170 def _apply_ml_ranking(
171     self,
172     ranked: list,
173     request: RecommendationRequestDto,
```

```
174         top_k: int,
175     ) -> list:
176         """
177         Applies LightGBM LambdaRank when trained, otherwise uses the fixed
178         semantic/collaborative/contextual fallback formula.
179         """
180
181         rescored = []
182
183         for recommendation in ranked:
184             destination = self._destination_by_id[recommendation.id]
185
186             semantic_score = float(
187                 getattr(recommendation.components, "semantic", 0.0) or 0.0
188             )
189
190             user_history_score = float(
191                 getattr(recommendation.components, "collaborative", 0.0) or 0.0
192             )
193
194             contextual_score = self._metadata_float(
195                 recommendation.metadata.get("contextual_score"),
196             )
197
198             features = build_ranker_features(
199                 semantic_score=semantic_score,
200                 collaborative_score=user_history_score,
201                 contextual_score=contextual_score,
202                 popularity_score=float(destination.popularity_score or 0.0),
203                 activity_match=float(
204                     getattr(recommendation.components, "activity_match", 0.0) or 0.0
205                 ),
206                 season_match=float(
207                     getattr(recommendation.components, "season_match", 0.0) or 0.0
208                 ),
209                 budget_match=float(
210                     getattr(recommendation.components, "budget_match", 0.0) or 0.0
211                 ),
212                 vibe_match=float(
213                     getattr(recommendation.components, "vibe_match", 0.0) or 0.0
214                 ),
215             )
216
217             ml_rank_score = self._ranker.predict_score(features)
218
219             recommendation.score = round(ml_rank_score, 4)
220             recommendation.metadata["ml_rank_score"] = f"{ml_rank_score:.4f}"
221             recommendation.metadata["ml_ranker_trained"] = str(
222                 self._ranker.is_trained
223             ).lower()
224             recommendation.metadata["ranker_mode"] = self._ranker.mode
225             recommendation.metadata["ranker_model"] = (
226                 "lightgbm_lambdarank"
227                 if self._ranker.is_trained
228                 else "fixed_weight_fallback"
229             )
230
231             rescored.append(recommendation)
232
```

```
233         rescored.sort(key=lambda item: item.score, reverse=True)
234         return rescored[:top_k]
235
236     def _build_semantic_only_response(
237         self,
238         candidates: list,
239         request: RecommendationRequestDto,
240     ) -> RecommendationResponseDto:
241         sorted_candidates = sorted(
242             candidates,
243             key=lambda c: c.get("semantic_score", 0.0),
244             reverse=True,
245         )[: request.top_k]
246
247         items = []
248
249         for candidate in sorted_candidates:
250             destination = candidate["destination"]
251             score = round(float(candidate.get("semantic_score", 0.0)), 4)
252
253             items.append(
254                 RecommendationResponseItemDto(
255                     id=destination.id,
256                     name=destination.name,
257                     district=destination.district,
258                     province=destination.province,
259                     score=score,
260                     components={"semantic": score},
261                     reasons=["Semantic baseline - no reranking applied"],
262                     metadata={"mode": "semantic_only"},
263                 )
264             )
265
266         return RecommendationResponseDto(results=items, total=len(items))
267
268     @staticmethod
269     def _to_dto(recommendation) -> RecommendationResponseItemDto:
270         return RecommendationResponseItemDto(
271             id=recommendation.id,
272             name=recommendation.name,
273             district=recommendation.district,
274             province=recommendation.province,
275             score=recommendation.score,
276             components=recommendation.components.model_dump(),
277             reasons=recommendation.reasons,
278             metadata=recommendation.metadata,
279         )
280
281     @staticmethod
282     def _is_cold_start(collab_scores: dict[str, float]) -> bool:
283         return all(score == 0.0 for score in collab_scores.values())
284
285     @staticmethod
286     def _metadata_float(value: str | None) -> float:
287         try:
288             return float(value or 0.0)
289         except (TypeError, ValueError):
290             return 0.0
```

M.3 Backend Contextual Reranker Excerpt

Listing M.3: Selected contextual reranking implementation.

```

1  from __future__ import annotations
2
3  from typing import Dict, List, Optional, Set
4
5  from backend.core.config import settings
6  from backend.core.constants import AccessibilityScores, BudgetOrder
7  from backend.domain.entities.accommodation import Accommodation
8  from backend.domain.entities.destination import Destination
9  from backend.domain.entities.recommendation import Recommendation, RecommendationComponents
10
11
12  class TextNormalizer:
13      def normalize(self, value: Optional[str]) -> str:
14          return value.strip().lower() if value else ""
15
16
17  class ContextualReranker:
18      """
19      Combines semantic, collaborative, and contextual signals into a final
20      recommendation score, then applies diversity constraints so that the
21      output is not dominated by one district or category.
22      """
23
24      def __init__(self) -> None:
25          self._norm = TextNormalizer()
26
27      def rerank(
28          self,
29          candidates: List[Dict],
30          accommodations: List[Accommodation],
31          collaborative_scores: Dict[str, float],
32          activity: str,
33          budget: str,
34          season: str,
35          vibe: str,
36          family_friendly: Optional[bool],
37          adventure_level: Optional[int],
38          top_k: int,
39      ) -> List[Recommendation]:
40          results: List[Recommendation] = []
41
42          for item in candidates:
43              destination: Destination = item["destination"]
44
45              semantic_score = self._clamp(item.get("semantic_score", 0.0))
46              collaborative_score = self._clamp(
47                  collaborative_scores.get(destination.id, 0.0)
48              )
49
50              activity_match = self._activity_match(destination, activity)
51              vibe_match = self._vibe_match(destination, vibe)
52              season_match = self._season_match(destination, season)
53              budget_match = self._budget_match(destination, budget)
54              accessibility_fit = self._accessibility_fit(destination)
55              family_fit = self._family_fit(destination, family_friendly)

```

```
56     accommodation_fit = self._accommodation_fit(
57         destination,
58         accommodations,
59         budget,
60     )
61
62     contextual_score = self._clamp(
63         activity_match * settings.activity_weight
64         + vibe_match * settings.vibe_weight
65         + season_match * settings.season_weight
66         + budget_match * settings.budget_weight
67         + accessibility_fit * settings.accessibility_weight
68         + family_fit * settings.family_weight
69         + accommodation_fit * settings.accommodation_weight
70     )
71
72     final_score = self._clamp(
73         0.50 * semantic_score
74         + 0.20 * collaborative_score
75         + 0.30 * contextual_score
76     )
77
78     results.append(
79         Recommendation(
80             id=destination.id,
81             name=destination.name,
82             district=destination.district,
83             province=destination.province,
84             score=round(final_score, 4),
85             components=RecommendationComponents(
86                 semantic=round(semantic_score, 4),
87                 collaborative=round(collaborative_score, 4),
88                 activity_match=round(activity_match, 4),
89                 vibe_match=round(vibe_match, 4),
90                 season_match=round(season_match, 4),
91                 budget_match=round(budget_match, 4),
92                 accessibility_fit=round(accessibility_fit, 4),
93                 family_fit=round(family_fit, 4),
94                 accommodation_fit=round(accommodation_fit, 4),
95             ),
96             reasons=[],
97             metadata={
98                 "district": destination.district or "",
99                 "province": destination.province or "",
100                 "budget_level": destination.budget_level or "",
101                 "accessibility": destination.accessibility or "",
102                 "primary_category": self._primary_category(destination),
103                 "contextual_score": f"{contextual_score:.4f}",
104                 "fixed_formula_score": f"{final_score:.4f}",
105                 "retrieval_score": f"{item.get('retrieval_score', 0.0):.4f}",
106             },
107         )
108     )
109
110     results.sort(key=lambda recommendation: recommendation.score, reverse=True)
111
112     return self._apply_diversity(
113         results,
114         top_k=top_k,
```

```
115         )
116
117     def _all_terms(self, destination: Destination) -> Set[str]:
118         terms = [
119             *destination.activities,
120             *destination.category,
121             *destination.tags,
122         ]
123
124         return {
125             self._norm.normalize(term)
126             for term in terms
127             if self._norm.normalize(term)
128         }
129
130     def _expanded_terms(self, value: str) -> Set[str]:
131         query = self._norm.normalize(value)
132
133         if not query:
134             return set()
135
136         alias_map = {
137             "trekking": {
138                 "hiking",
139                 "trail",
140                 "trek",
141                 "high altitude pass crossing",
142                 "acclimatization",
143                 "mountaineering",
144             },
145             "hiking": {
146                 "trekking",
147                 "trail",
148                 "trek",
149             },
150             "adventure": {
151                 "trekking",
152                 "hiking",
153                 "high altitude pass crossing",
154                 "mountaineering",
155                 "expedition",
156                 "rafting",
157                 "paragliding",
158                 "zipline",
159             },
160             "culture": {
161                 "cultural",
162                 "heritage",
163                 "traditional",
164                 "exploring walled city",
165                 "palace tours",
166                 "local crafts",
167                 "museum",
168             },
169             "cultural": {
170                 "culture",
171                 "heritage",
172                 "traditional",
173                 "historic",
```

```
174         "exploring walled city",
175         "museum",
176     },
177     "historic": {
178         "heritage",
179         "cultural",
180         "traditional",
181     },
182     "photography": {
183         "viewpoint",
184         "scenic",
185         "panorama",
186         "sunrise",
187     },
188     "viewpoint": {
189         "photography",
190         "scenic",
191         "panorama",
192         "sunrise",
193     },
194     "boating": {
195         "lake",
196         "waterside",
197         "kayaking",
198     },
199     "lake": {
200         "boating",
201         "waterside",
202         "scenic",
203     },
204     "pilgrimage": {
205         "spiritual",
206         "temple",
207         "heritage",
208         "monastery visit",
209         "religious",
210         "shrine",
211         "sacred",
212         "cave exploration",
213         "cultural study",
214     },
215     "spiritual": {
216         "pilgrimage",
217         "temple",
218         "monastery visit",
219         "religious",
220         "shrine",
221         "sacred",
222     },
223     "relaxation": {
224         "quiet",
225         "peaceful",
226         "retreat",
227         "swimming",
228         "fishing",
229         "picnic",
230         "boating",
231     },
232     "peaceful": {
```

```
233         "quiet",
234         "relaxation",
235         "retreat",
236         "tranquil",
237         "serene",
238     },
239     "quiet": {
240         "peaceful",
241         "relaxation",
242         "retreat",
243         "tranquil",
244         "serene",
245     },
246     "nature": {
247         "wildlife",
248         "outdoors",
249         "scenic",
250         "bird watching",
251         "nature walk",
252         "forest",
253     },
254     "wildlife": {
255         "nature",
256         "forest",
257         "bird watching",
258         "conservation",
259     },
260     "family": {
```

M.4 Flutter Routing Service

Listing M.4: Implemented Flutter routing service with fallback tiers.

```
1 // OSRM public demo server is free but has rate limits.
2 // For production at scale, self-host OSRM or switch to:
3 //   - OpenRouteService (free tier: 2000 req/day)
4 //   - Valhalla (fully open source, self-hostable)
5 //   - GraphHopper (free tier available)
6 // No API key is required for the current implementation.
7
8 import 'dart:convert';
9
10 import 'package:http/http.dart' as http;
11 import 'package:latlong2/latlong.dart';
12
13 import 'package:rural_tourism_app/core/navigation/models/route_result.dart';
14 import 'package:rural_tourism_app/core/navigation/route_cache.dart';
15 import 'package:rural_tourism_app/core/utils/haversine.dart';
16
17 class RoutingService {
18   final http.Client _client;
19   final RouteCache _cache;
20
21   RoutingService({
22     http.Client? client,
23     RouteCache? cache,
24   }) : _client = client ?? http.Client(),
25       _cache = cache ?? RouteCache.instance;
26
27   Future<RouteResult?> getRoute(
28     LatLng origin,
29     LatLng destination, {
30     TravelMode travelMode = TravelMode.driving,
31   }) async {
32     final cacheKey = _cache.buildKey(origin, destination, travelMode);
33     final cachedRoute = await _cache.get(cacheKey);
34     if (cachedRoute != null) return cachedRoute;
35
36     final onlineRoute = await _getOnlineRoute(
37       origin,
38       destination,
39       travelMode: travelMode,
40     );
41     if (onlineRoute != null) {
42       await _cache.save(cacheKey, onlineRoute);
43       return onlineRoute;
44     }
45
46     return _fallbackRoute(
47       origin,
48       destination,
49       travelMode: travelMode,
50     );
51   }
52
53   Future<RouteResult?> _getOnlineRoute(
54     LatLng origin,
55     LatLng destination, {
```

```
56     required TravelMode travelMode,
57 }) async {
58     try {
59         final uri = Uri.parse(
60             'http://router.project-osrm.org/route/v1/${travelMode.osrmPath}/'
61             '${origin.longitude},${origin.latitude};'
62             '${destination.longitude},${destination.latitude}'
63             '?overview=full&geometries=geojson&steps=true&annotations=false',
64         );
65
66         final response = await _client.get(uri).timeout(
67             const Duration(seconds: 12),
68         );
69
70         if (response.statusCode < 200 || response.statusCode >= 300) {
71             return null;
72         }
73
74         final body = jsonDecode(response.body) as Map<String, dynamic>;
75         if (body['code'] != 'Ok') return null;
76
77         final routes = body['routes'] as List<dynamic>?;
78         if (routes == null || routes.isEmpty) return null;
79
80         final route = routes.first as Map<String, dynamic>;
81         final geometry = route['geometry'] as Map<String, dynamic>?;
82         final coordinates = geometry?['coordinates'] as List<dynamic>?;
83         if (coordinates == null || coordinates.isEmpty) return null;
84
85         final points = coordinates
86             .map((coordinate) {
87                 final pair = coordinate as List<dynamic>;
88                 return LatLng(
89                     (pair[1] as num).toDouble(),
90                     (pair[0] as num).toDouble(),
91                 );
92             })
93             .where((point) => point.latitude.isFinite && point.longitude.isFinite)
94             .toList(growable: false);
95
96         if (points.isEmpty) return null;
97
98         final steps = <RouteStep>[];
99         final legs = route['legs'] as List<dynamic>? ?? const [];
100         for (final leg in legs) {
101             final legSteps =
102                 (leg as Map<String, dynamic>)['steps'] as List<dynamic>? ??
103                 const [];
104             for (final rawStep in legSteps) {
105                 final step = rawStep as Map<String, dynamic>;
106                 final maneuver = step['maneuver'] as Map<String, dynamic>? ?? {};
107                 final location = maneuver['location'] as List<dynamic>?;
108                 if (location == null || location.length < 2) continue;
109
110                 final type = (maneuver['type'] as String?) ?? '';
111                 final modifier = (maneuver['modifier'] as String?) ?? '';
112                 final name = (step['name'] as String?)?.trim() ?? '';
113
114                 steps.add(
```

```
115         RouteStep(  
116             instruction: _instructionFor(type, modifier, name),  
117             distanceMeters: ((step['distance'] as num?) ?? 0).toDouble(),  
118             durationSeconds: ((step['duration'] as num?) ?? 0).toDouble(),  
119             maneuverType: type,  
120             maneuverDirection: modifier,  
121             location: LatLng(  
122                 (location[1] as num).toDouble(),  
123                 (location[0] as num).toDouble(),  
124             ),  
125         ),  
126     );  
127 }  
128 }  
129  
130     return RouteResult(  
131         polylinePoints: points,  
132         steps: steps,  
133         distanceMeters: ((route['distance'] as num?) ?? 0).toDouble(),  
134         durationSeconds: ((route['duration'] as num?) ?? 0).toDouble(),  
135         travelMode: travelMode,  
136     );  
137 } catch (_) {  
138     return null;  
139 }  
140 }  
141  
142 RouteResult _fallbackRoute(  
143     LatLng origin,  
144     LatLng destination, {  
145         required TravelMode travelMode,  
146     }) {  
147         final distanceMeters = haversineKm(  
148             origin.latitude,  
149             origin.longitude,  
150             destination.latitude,  
151             destination.longitude,  
152         ) *  
153         1000;  
154         final speedKmh = switch (travelMode) {  
155             TravelMode.driving => 60.0,  
156             TravelMode.walking => 5.0,  
157             TravelMode.cycling => 15.0,  
158         };  
159         final durationSeconds = distanceMeters / (speedKmh * 1000 / 3600);  
160  
161         return RouteResult(  
162             polylinePoints: [origin, destination],  
163             steps: [  
164                 RouteStep(  
165                     instruction: 'Approximate route - internet required for turn-by-turn',  
166                     distanceMeters: distanceMeters,  
167                     durationSeconds: durationSeconds,  
168                     maneuverType: 'fallback',  
169                     maneuverDirection: 'straight',  
170                     location: destination,  
171                 ),  
172             ],  
173             distanceMeters: distanceMeters,
```

```
174     durationSeconds: durationSeconds,
175     travelMode: travelMode,
176     isFallback: true,
177 );
178 }
179
180 String _instructionFor(String type, String modifier, String name) {
181     final direction = _readableDirection(modifier);
182     final road = name.isEmpty ? '' : ' onto $name';
183
184     return switch (type) {
185         'depart' => 'Head ${direction.isEmpty ? 'straight' : direction}$road',
186         'arrive' => 'Arrive at destination',
187         'turn' => 'Turn ${direction.isEmpty ? 'ahead' : direction}$road',
188         'new name' => 'Continue$road',
189         'continue' =>
190             'Continue ${direction.isEmpty ? 'straight' : direction}$road',
191         'merge' => 'Merge ${direction.isEmpty ? 'ahead' : direction}$road',
192         'on ramp' => 'Take the ramp$road',
193         'off ramp' => 'Take the exit$road',
194         'fork' => 'Keep ${direction.isEmpty ? 'ahead' : direction}$road',
195         'end of road' =>
196             'At the end of the road, go ${direction.isEmpty ? 'ahead' : direction}$road',
197         'roundabout' || 'rotary' => 'Enter the roundabout$road',
198         'roundabout turn' =>
199             'At the roundabout, turn ${direction.isEmpty ? 'ahead' : direction}$road',
200         'notification' => 'Continue$road',
201         _ => 'Continue ${direction.isEmpty ? 'ahead' : direction}$road',
202     };
203 }
204
205 String _readableDirection(String modifier) {
206     return switch (modifier) {
207         'uturn' => 'around',
208         'sharp right' => 'sharp right',
209         'right' => 'right',
210         'slight right' => 'slight right',
211         'straight' => 'straight',
212         'slight left' => 'slight left',
213         'left' => 'left',
214         'sharp left' => 'sharp left',
215         _ => '',
216     };
217 }
218 }
```

M.5 Offline Semantic Encoder

Listing M.5: Implemented offline semantic encoding support.

```

1 import 'dart:math';
2
3 import 'package:rural_tourism_app/features/destinations/domain/models/destination.dart';
4 import 'package:rural_tourism_app/features/recommendations/domain/models/user_preferences.dart';
5
6 abstract final class OfflineSemanticEncoder {
7   static const String modelName = 'gandaki-offline-semantic-v1';
8   static const int dimension = 96;
9   static const int _conceptDimensions = 28;
10
11   static const _stopWords = {
12     'a',
13     'an',
14     'and',
15     'are',
16     'as',
17     'at',
18     'by',
19     'for',
20     'from',
21     'in',
22     'into',
23     'is',
24     'it',
25     'near',
26     'of',
27     'on',
28     'or',
29     'the',
30     'to',
31     'with',
32   };
33
34   static const List<List<String>> _conceptLexicon = [
35     ['trekking', 'hiking', 'trail', 'ridge', 'pass', 'base camp', 'mountain'],
36     ['adventure', 'climbing', 'cave', 'high altitude', 'expedition', 'thrill'],
37     ['culture', 'cultural', 'heritage', 'museum', 'traditional', 'history'],
38     ['village', 'homestay', 'community', 'local food', 'terrace', 'rural'],
39     ['pilgrimage', 'temple', 'spiritual', 'dham', 'gumba', 'monastery'],
40     ['nature', 'forest', 'river', 'valley', 'landscape', 'green hills'],
41     ['wildlife', 'bird', 'birding', 'conservation', 'wetland', 'jungle'],
42     ['lake', 'boating', 'waterside', 'pond', 'waterfall', 'hot spring'],
43     ['scenic', 'viewpoint', 'panorama', 'sunrise', 'photography', 'view'],
44     ['family', 'safe', 'easy', 'picnic', 'children', 'accessible'],
45     ['peaceful', 'quiet', 'relax', 'retreat', 'calm', 'slow travel'],
46     ['budget', 'low cost', 'basic rooms', 'affordable', 'homestay'],
47     ['medium', 'guesthouse', 'lodge', 'private rooms', 'comfort'],
48     ['premium', 'hotel', 'resort', 'luxury', 'comfort'],
49     ['spring', 'rhododendron', 'flowers', 'clear weather'],
50     ['autumn', 'festival', 'clear sky', 'mountain view'],
51     ['winter', 'cool weather', 'clear view', 'snow'],
52     ['monsoon', 'summer', 'green hills', 'rainy season'],
53     ['food', 'market', 'bazaar', 'apple', 'local meals'],
54     ['gorkha', 'durbar', 'palace', 'fort', 'historic'],
55     ['mustang', 'manang', 'himalaya', 'trans himalaya', 'dry valley'],

```

```
56      ['pokhara', 'kaski', 'phewa', 'begnas', 'sarangkot'],
57      ['nawalpur', 'lowland', 'plain', 'tharu', 'community forest'],
58      ['myagdi', 'baglung', 'parbat', 'hot spring', 'kalika'],
59      ['lamjung', 'marsyangdi', 'gurung', 'ghalegaun'],
60      ['tanahun', 'bandipur', 'cave', 'seti', 'damauli'],
61      ['syangja', 'aandhikhola', 'hill', 'orange', 'palace'],
62      ['accessibility', 'easy', 'moderate', 'difficult', 'road', 'walk'],
63  ];
64
65  static List<double> encodeDestination(Destination destination) {
66      final vector = List<double>.filled(dimension, 0.0);
67
68      _addText(vector, destination.name, 3.2);
69      _addText(vector, destination.district ?? '', 1.6);
70      _addText(vector, destination.municipality ?? '', 1.1);
71      _addText(vector, destination.category.join(' '), 4.2);
72      _addText(vector, destination.activities.join(' '), 4.0);
73      _addText(vector, destination.tags.join(' '), 2.8);
74      _addText(vector, destination.shortDescription, 1.8);
75      _addText(vector, destination.fullDescription, 1.2);
76      _addText(vector, destination.priceTier, 1.2);
77      _addText(vector, destination.accessibility ?? '', 1.0);
78      _addText(vector, destination.bestSeason.join(' '), 1.4);
79
80      if (destination.familyFriendly == true) {
81          _addText(vector, 'family safe easy accessible', 2.0);
82      }
83
84      _addLevel(vector, 0, destination.adventureLevel, 0.28);
85      _addLevel(vector, 2, destination.cultureLevel, 0.28);
86      _addLevel(vector, 5, destination.natureLevel, 0.28);
87
88      return _l2Normalize(vector);
89  }
90
91  static List<double> encodePreferences(
92      UserPreferences prefs, {
93      bool? familyFriendly,
94      int? adventureLevel,
95  }) {
96      final vector = List<double>.filled(dimension, 0.0);
97
98      _addText(vector, prefs.activity, 5.0);
99      _addText(vector, _expandedActivity(prefs.activity), 3.5);
100     _addText(vector, prefs.vibe, 4.0);
101     _addText(vector, _expandedVibe(prefs.vibe), 2.8);
102     _addText(vector, prefs.budget, 2.0);
103     _addText(vector, prefs.season, 1.8);
104
105     if (familyFriendly == true) {
106         _addText(vector, 'family safe easy accessible homestay', 2.6);
107     } else if (familyFriendly == false) {
108         _addText(vector, 'adventure trail difficult independent', 1.4);
109     }
110
111     if (adventureLevel != null) {
112         if (adventureLevel >= 4) {
113             _addText(vector, 'trekking adventure pass ridge high altitude', 2.8);
114         } else if (adventureLevel <= 2) {
```

```
115         _addText(vector, 'easy family village lake peaceful', 2.2);
116     } else {
117         _addText(vector, 'moderate hike scenic village nature', 1.8);
118     }
119 }
120
121 return _l2Normalize(vector);
122 }
123
124 static void _addText(List<double> vector, String input, double weight) {
125     final normalized = _normalize(input);
126     if (normalized.isEmpty || weight <= 0) {
127         return;
128     }
129
130     _addConcepts(vector, normalized, weight);
131
132     final tokens = _tokens(normalized);
133     for (final token in tokens) {
134         _addHashFeature(vector, token, weight);
135     }
136
137     for (var index = 0; index < tokens.length - 1; index++) {
138         _addHashFeature(vector, '${tokens[index]} ${tokens[index + 1]}', weight);
139     }
140 }
141
142 static void _addConcepts(List<double> vector, String text, double weight) {
143     for (var index = 0; index < _conceptLexicon.length; index++) {
144         var conceptScore = 0.0;
145         for (final phrase in _conceptLexicon[index]) {
146             final normalizedPhrase = _normalize(phrase);
147             if (text == normalizedPhrase) {
148                 conceptScore += 1.0;
149             } else if (text.contains(normalizedPhrase)) {
150                 conceptScore += 0.82;
151             }
152         }
153         if (conceptScore > 0) {
154             vector[index] += weight * min(conceptScore, 2.4);
155         }
156     }
157 }
158
159 static void _addLevel(
160     List<double> vector,
161     int conceptIndex,
162     int? value,
163     double weight,
164 ) {
165     if (value == null || conceptIndex >= _conceptDimensions) {
166         return;
167     }
168     vector[conceptIndex] += (value.clamp(1, 5) / 5.0) * weight;
169 }
170
171 static void _addHashFeature(
172     List<double> vector, String token, double weight) {
173     if (token.isEmpty) return;
```

```
174     final hash = _stableHash(token);
175     final index = _conceptDimensions + hash % (dimension - _conceptDimensions);
176     final strength = 0.35 + ((hash >> 8) & 0xff) / 255.0 * 0.65;
177     vector[index] += weight * strength;
178 }
179
180 static List<String> _tokens(String input) {
181     return input
182         .split(RegExp(r'\s+'))
183         .where((token) => token.length > 1 && !_stopWords.contains(token))
184         .toList();
185 }
186
187 static String _expandedActivity(String activity) {
188     switch (_normalize(activity)) {
189         case 'trekking':
190         case 'hiking':
191             return 'trekking hiking trail ridge mountain pass viewpoint adventure';
192         case 'adventure':
193             return 'adventure trekking climbing cave pass high altitude thrill';
194         case 'culture':
195         case 'cultural':
196             return 'culture heritage village homestay traditional local food';
197         case 'pilgrimage':
198             return 'pilgrimage temple spiritual dham gumba monastery';
199         case 'wildlife':
200             return 'wildlife birding forest conservation wetland nature';
201         case 'boating':
202         case 'lake':
203             return 'lake boating waterside scenic family relaxation';
204         case 'photography':
205             return 'photography viewpoint panorama scenic sunrise mountain view';
206         case 'relaxation':
207             return 'relaxation peaceful quiet retreat lake village hot spring';
208         default:
209             return activity;
210     }
211 }
212
213 static String _expandedVibe(String vibe) {
214     switch (_normalize(vibe)) {
215         case 'family':
216             return 'family safe easy accessible picnic homestay';
217         case 'adventure':
218             return 'adventure trekking trail climbing pass';
219         case 'cultural':
220         case 'historic':
221             return 'culture heritage traditional history market village';
222         case 'spiritual':
223             return 'spiritual pilgrimage temple monastery peaceful';
224         case 'nature':
225             return 'nature forest river lake valley wildlife';
226         case 'scenic':
227             return 'scenic viewpoint panorama photography sunrise';
228         case 'peaceful':
229         case 'quiet':
230             return 'peaceful quiet retreat relax village slow travel';
231         default:
232             return vibe;
```

```
233     }
234 }
235
236 static String _normalize(String input) {
237     return input
238         .toLowerCase()
239         .replaceAll(RegExp(r'^a-z0-9\s'), ' ')
240         .trim()
241         .replaceAll(RegExp(r'\s+'), ' ');
242 }
243
244 static int _stableHash(String input) {
245     const fnvPrime = 16777619;
246     var hash = 2166136261;
247     for (final codeUnit in input.codeUnits) {
248         hash ^= codeUnit;
249         hash = (hash * fnvPrime) & 0xffffffff;
250     }
251     return hash & 0x7fffffff;
252 }
253
254 static List<double> _l2Normalize(List<double> vector) {
255     final magnitude = sqrt(
256         vector.fold(0.0, (sum, value) => sum + value * value),
257     );
258     if (magnitude == 0) {
259         return vector;
260     }
261     return vector.map((value) => value / magnitude).toList();
262 }
263 }
```

M.6 Intelligence Orchestrator Excerpt

Listing M.6: Selected orchestration logic for local and online intelligence.

```

1  import 'dart:convert';
2
3  import 'package:flutter/services.dart';
4
5  import 'package:rural_tourism_app/features/intelligence/core/intelligence_config.dart';
6  import 'package:rural_tourism_app/features/intelligence/core/intelligence_constants.dart';
7  import 'package:rural_tourism_app/features/intelligence/dialogue/clarification_generator.dart';
8  import 'package:rural_tourism_app/features/intelligence/dialogue/conversation_memory.dart';
9  import 'package:rural_tourism_app/features/intelligence/dialogue/dialogue_manager.dart';
10 import 'package:rural_tourism_app/features/intelligence/dialogue/dialogue_state_tracker.dart';
11 import 'package:rural_tourism_app/features/intelligence/dialogue/slot_filling_manager.dart';
12 import 'package:rural_tourism_app/features/intelligence/embeddings/embedding_encoder.dart';
13 import 'package:rural_tourism_app/features/intelligence/embeddings/embedding_index.dart';
14 import 'package:rural_tourism_app/features/intelligence/embeddings/semantic_retriever.dart';
15 import 'package:rural_tourism_app/features/intelligence/intent/hybrid_intent_classifier.dart';
16 import 'package:rural_tourism_app/features/intelligence/intent/intent_training_data.dart';
17 import 'package:rural_tourism_app/features/intelligence/intent/semantic_intent_classifier.dart';
18 import 'package:rural_tourism_app/features/intelligence/models/chatbot_request.dart';
19 import 'package:rural_tourism_app/features/intelligence/models/chatbot_response.dart';
20 import 'package:rural_tourism_app/features/intelligence/models/entity_mention.dart';
21 import 'package:rural_tourism_app/features/intelligence/models/intent_classification_result.dart';
22 import 'package:rural_tourism_app/features/intelligence/models/language_detection_result.dart';
23 import 'package:rural_tourism_app/features/intelligence/models/translation_request.dart';
24 import 'package:rural_tourism_app/features/intelligence/models/translation_response.dart';
25 import 'package:rural_tourism_app/features/intelligence/nlp/advanced_tokenizer.dart';
26 import 'package:rural_tourism_app/features/intelligence/nlp/devanagari_normalizer.dart';
27 import 'package:rural_tourism_app/features/intelligence/nlp/language_detector.dart';
28 import 'package:rural_tourism_app/features/intelligence/nlp/named_entity_recognizer.dart';
29 import 'package:rural_tourism_app/features/intelligence/nlp/nepali_stemmer.dart';
30 import 'package:rural_tourism_app/features/intelligence/nlp/nlp_pipeline.dart';
31 import 'package:rural_tourism_app/features/intelligence/nlp/romanized_nepali_normalizer.dart';
32 import 'package:rural_tourism_app/features/intelligence/nlp/stopword_remover.dart';
33 import 'package:rural_tourism_app/features/intelligence/nlp/synonym_expander.dart';
34 import 'package:rural_tourism_app/features/intelligence/rag/context_retriever.dart';
35 import 'package:rural_tourism_app/features/intelligence/rag/rag_pipeline.dart';
36 import 'package:rural_tourism_app/features/intelligence/rag/response_generator.dart';
37 import 'package:rural_tourism_app/features/intelligence/rag/template_generator.dart';
38 import 'package:rural_tourism_app/features/intelligence/retrieval/bm25_retriever.dart';
39 import 'package:rural_tourism_app/features/intelligence/retrieval/hybrid_retriever.dart';
40 import 'package:rural_tourism_app/features/intelligence/retrieval/knowledge_repository.dart';
41 import 'package:rural_tourism_app/features/intelligence/retrieval/retrieval_reranker.dart';
42 import 'package:rural_tourism_app/features/intelligence/safety/emergency_detector.dart';
43 import 'package:rural_tourism_app/features/intelligence/safety/emergency_response_repository.dart';
44 import 'package:rural_tourism_app/features/intelligence/safety/safety_layer.dart';
45 import 'package:rural_tourism_app/features/intelligence/translation/glossary_translator.dart';
46 import 'package:rural_tourism_app/features/intelligence/translation/hybrid_translation_manager.dart';
47 import 'package:rural_tourism_app/features/intelligence/translation/neural_translation_adapter.dart';
48 import 'package:rural_tourism_app/features/intelligence/translation/phrasebook_translator.dart';
49 import 'package:rural_tourism_app/features/intelligence/translation/template_translator.dart';
50 import 'package:rural_tourism_app/features/intelligence/services/optional_gemini_service.dart';
51
52 class IntelligenceOrchestrator {
53   final IntelligenceConfig config;
54
55   bool _initialized = false;

```

```
56
57 late final KnowledgeRepository knowledgeRepository;
58 late final NlpPipeline nlpPipeline;
59 late final HybridIntentClassifier intentClassifier;
60 late final DialogueManager dialogueManager;
61 late final RagPipeline ragPipeline;
62 late final SafetyLayer safetyLayer;
63 late final HybridTranslationManager translationManager;
64 late final OptionalGeminiService optionalGeminiService;
65 late final Map<String, Map<String, dynamic>> _destinationById;
66
67 IntelligenceOrchestrator({this.config = IntelligenceConfig.production});
68
69 Future<void> initialize() async {
70     if (_initialized) return;
71
72     final romanizedDictionary = await _loadStringMap(
73         IntelligenceConstants.romanizedDictionaryAsset,
74     );
75     final stopwords = await _loadStringSet(
76         IntelligenceConstants.nepaliStopwordsAsset,
77         key: 'stopwords',
78     );
79     final ontology = await _loadOntology();
80     final destinationRecords = await _loadDestinationRecords();
81     _destinationById = {
82         for (final item in destinationRecords)
83             if ((item['id']?.toString() ?? '').isEmpty)
84                 item['id'].toString(): item,
85     };
86     final destinationGazetteer = _buildDestinationGazetteer(destinationRecords);
87     final accommodationGazetteer = await _loadAccommodationGazetteer();
88
89     knowledgeRepository = KnowledgeRepository();
90     await knowledgeRepository.load();
91
92     nlpPipeline = NlpPipeline(
93         languageDetector: LanguageDetector(
94             romanizedDictionary: romanizedDictionary.keys.toSet(),
95         ),
96         devanagariNormalizer: const DevanagariNormalizer(),
97         romanizedNormalizer: RomanizedNepaliNormalizer(
98             mappings: romanizedDictionary,
99         ),
100         tokenizer: const AdvancedTokenizer(),
101         stemmer: NepaliStemmer(
102             protectedTerms: destinationGazetteer.keys.toSet(),
103         ),
104         stopwordRemover: StopwordRemover(nepaliStopwords: stopwords),
105         namedEntityRecognizer: NamedEntityRecognizer(
106             destinationGazetteer: destinationGazetteer,
107             accommodationGazetteer: accommodationGazetteer,
108         ),
109         synonymExpander: SynonymExpander(ontology: ontology),
110     );
111
112     final encoder = const EmbeddingEncoder();
113     final embeddingIndex = EmbeddingIndex(encoder: encoder)
114         ..build(knowledgeRepository.entries);
```

```
115     final semanticRetriever = SemanticRetriever(  
116         encoder: encoder,  
117         index: embeddingIndex,  
118     );  
119     final bm25Retriever = BM25Retriever()..build(knowledgeRepository.entries);  
120     final hybridRetriever = HybridRetriever(  
121         semanticRetriever: semanticRetriever,  
122         bm25Retriever: bm25Retriever,  
123         config: config,  
124     );  
125  
126     Future<IntentTrainingData> trainingLoader() => IntentTrainingData.load();  
127     final semanticIntent = SemanticIntentClassifier(  
128         encoder: encoder,  
129         trainingDataLoader: trainingLoader,  
130     );  
131     intentClassifier = HybridIntentClassifier(  
132         semanticClassifier: semanticIntent,  
133         trainingDataLoader: trainingLoader,  
134     );  
135     await intentClassifier.load();  
136  
137     dialogueManager = DialogueManager(  
138         memory: ConversationMemory(config: config),  
139         slotFillingManager: const SlotFillingManager(),  
140         clarificationGenerator: const ClarificationGenerator(),  
141         stateTracker: const DialogueStateTracker(),  
142     );  
143  
144     ragPipeline = RagPipeline(  
145         contextRetriever: ContextRetriever(  
146             hybridRetriever: hybridRetriever,  
147             reranker: const RetrievalReranker(),  
148         ),  
149         responseGenerator: const ResponseGenerator(  
150             templateGenerator: TemplateGenerator(),  
151         ),  
152     );  
153  
154     safetyLayer = const SafetyLayer(  
155         detector: EmergencyDetector(),  
156         responseRepository: EmergencyResponseRepository(),  
157     );  
158  
159     translationManager = HybridTranslationManager(  
160         phrasebookTranslator: PhrasebookTranslator(),  
161         templateTranslator: TemplateTranslator(),  
162         glossaryTranslator: GlossaryTranslator(),  
163         neuralTranslationAdapter: NeuralTranslationAdapter(),  
164     );  
165     await translationManager.load();  
166     optionalGeminiService = OptionalGeminiService(config: config);  
167  
168     _initialized = true;  
169 }  
170  
171 Future<ChatbotResponse> respond(ChatbotRequest request) async {  
172     await initialize();  
173     final nlp = nlpPipeline.process(request.text);
```

```
174
175     final safety = safetyLayer.check(request.text, nlp.language);
176     if (safety != null) return safety;
177
178     final destinationMatch = _bestDestinationMatch(nlp.entities);
179     if (destinationMatch != null &&
180         _shouldAnswerDestinationDirectly(request.text, destinationMatch)) {
181         final response = _directDestinationResponse(
182             destinationMatch,
183             nlp.language,
184         );
185         dialogueManager.completeTurn(
186             conversationId: request.conversationId,
187             userText: request.text,
188             assistantText: response.text,
189             intent: response.intent,
190             confidence: response.confidence,
191         );
192         return response;
193     }
194
195     final intent = intentClassifier.classify(nlp);
196     final dialogue = dialogueManager.updateBeforeResponse(
197         conversationId: request.conversationId,
198         nlp: nlp,
199         intent: intent,
200     );
201
202     final rag = ragPipeline.run(
203         nlp: nlp,
204         intent: intent.intent,
205         dialogueState: dialogue.state,
206     );
207     final shouldAppendClarification = _shouldAppendLowConfidenceClarification(
208         intent.intent, intent.confidence);
209     var text = rag.text;
210     var source = ChatbotResponseSource.offlineKnowledgeBase;
211     if (shouldAppendClarification && rag.contexts.isNotEmpty) {
212         text = _appendLowConfidenceClarification(text);
213     } else if (rag.contexts.isEmpty &&
214         destinationMatch == null &&
215         !_hasForcedKeywordTrigger(intent) &&
216         intent.confidence < 0.28 &&
217         dialogue.shouldClarify &&
218         dialogue.clarificationQuestion != null) {
219         text = dialogue.clarificationQuestion!;
220         source = ChatbotResponseSource.offlineModel;
221     }
222     final combinedConfidence =
223         (intent.confidence * 0.45 + rag.confidence * 0.55).clamp(0.0, 1.0);
224
225     if (request.allowOnlineEnhancement) {
226         final enhanced = await optionalGeminiService.enhance(
227             userMessage: request.text,
228             localAnswer: text,
229             isEmergency: false,
230             localConfidence: combinedConfidence,
231         );
232         if (enhanced != null) {
```

```
233     text = enhanced;
234     source = ChatbotResponseSource.onlineEnhancement;
235     dialogueManager.completeTurn(
236         conversationId: request.conversationId,
237         userText: request.text,
238         assistantText: text,
239         intent: intent.intent,
240         confidence: 0.95,
241     );
242     return ChatbotResponse(
243         text: text,
244         intent: intent.intent,
245         confidence: 0.95,
246         isEmergency: false,
247         source: source,
248         language: nlp.language,
249         intentResult: intent,
250         suggestions: _suggestionsForIntent(intent.intent),
251     );
252 }
253 }
254
255 dialogueManager.completeTurn(
256     conversationId: request.conversationId,
257     userText: request.text,
258     assistantText: text,
259     intent: intent.intent,
260     confidence: combinedConfidence,
261 );
262
263 return ChatbotResponse(
264     text: text,
265     intent: intent.intent,
266     confidence: combinedConfidence,
267     isEmergency: false,
268     source: source,
269     language: nlp.language,
270     intentResult: intent,
271     retrievedContexts: rag.contexts,
272     suggestions: rag.suggestions,
273     metadata: {
274         'rag_method': rag.method,
275         'matched_features': intent.matchedFeatures,
276     },
277 );
278 }
279
280 Future<TranslationResponse> translate(TranslationRequest request) async {
281     await initialize();
282     return translationManager.translate(request);
283 }
284
285 List<String> _suggestionsForIntent(String intent) {
286     switch (intent) {
287         case 'food_query':
288             return ['Best local food', 'Vegetarian options', 'Dal bhat places'];
289         case 'homestay_query':
290         case 'homestay_search':
291             return [
```

```
292         'Find homestay nearby',
293         'Budget accommodation',
294         'Village stays'
295     ];
296     case 'safety_info':
297     case 'safety_concern':
298         return ['Trekking safety', 'Permit info', 'Emergency contacts'];
299     case 'transport_query':
300         return ['Bus routes', 'Jeep hire', 'Walking distance'];
301     case 'best_time_to_visit':
302         return ['Spring season', 'Autumn trekking', 'Monsoon travel'];
303     case 'trekking_info':
304     case 'adventure_activity':
305     case 'trekking':
306         return ['Easy treks', 'Popular routes', 'Permit info'];
307     default:
308         return ['Destinations', 'Homestays', 'Food & Culture'];
309     }
310 }
311
312 bool _shouldAppendLowConfidenceClarification(
313     String intent,
314     double confidence,
315 ) {
316     return intent != 'fallback' && confidence >= 0.28 && confidence < 0.55;
317 }
318
319 bool _hasForcedKeywordTrigger(IntentClassificationResult intent) {
320     return intent.matchedFeatures.any(
```

M.7 Backend Collaborative Filtering Test

Listing M.7: Automated collaborative filtering tests.

```
1 from backend.core.constants import EventTypes
2 from backend.domain.entities.interaction import Interaction
3 from backend.infrastructure.ml.collaborative_filter import CollaborativeFilter
4
5
6 def test_neutral_impressions_do_not_boost_collaborative_or_popularity_scores():
7     recommender = CollaborativeFilter(
8         [
9             Interaction(
10                 user_id="user-1",
11                 destination_id="dest-neutral",
12                 event_type=EventTypes.RECOMMENDATION_SHOWN,
13                 value=1.0,
14             ),
15             Interaction(
16                 user_id="user-2",
17                 destination_id="dest-positive",
18                 event_type=EventTypes.CLICK,
19                 value=1.0,
20             ),
21         ]
22     )
23
24     collaborative_scores = recommender.score_candidates(
25         user_id="user-1",
26         candidate_ids=["dest-positive"],
27     )
28     popularity_scores = recommender.popular_destinations(
29         candidate_ids=["dest-neutral", "dest-positive"],
30     )
31
32     assert collaborative_scores["dest-positive"] == 0.0
33     assert popularity_scores["dest-neutral"] == 0.0
34     assert popularity_scores["dest-positive"] == 1.0
```

M.8 Flutter Intelligence End-to-End Test

Listing M.8: Automated local intelligence end-to-end tests.

```
1 import 'package:flutter_test/flutter_test.dart';
2 import 'package:rural_tourism_app/features/intelligence/models/chatbot_request.dart';
3 import 'package:rural_tourism_app/features/intelligence/models/chatbot_response.dart';
4 import 'package:rural_tourism_app/features/intelligence/services/intelligence_orchestrator.dart';
5
6 void main() {
7   TestWidgetsFlutterBinding.ensureInitialized();
8
9   test('emergency chatbot response bypasses online enhancement', () async {
10     final orchestrator = IntelligenceOrchestrator();
11     await orchestrator.initialize();
12
13     final response = await orchestrator.respond(
14       const ChatbotRequest(
15         text: 'I am injured and need ambulance help',
16         conversationId: 'emergency_test',
17         allowOnlineEnhancement: true,
18       ),
19     );
20
21     expect(response.isEmergency, isTrue);
22     expect(response.intent, 'emergency_help');
23     expect(response.source, ChatbotResponseSource.emergencyProtocol);
24     expect(response.text, contains('Ambulance'));
25     expect(response.metadata['contacts'], isNotEmpty);
26   });
27 }
```

Appendix N

Project Governance, Risk, and Evidence Index

Table N.1: Project risk register.

ID	Risk	Prob.	Impact	Control
R-01	Rural connectivity prevents online maps, routes, or AI responses.	High	High	Local assets, caches, route fallback, and local intelligence.
R-02	Curated destination information becomes stale.	Medium	High	Versioned dataset, validation, update governance, and visible limitations.
R-03	Haversine fallback is misunderstood as a safe road route.	Medium	High	Explicit source labelling and warning language.
R-04	Recommendation results reinforce popularity bias.	Medium	Medium	Diversity control, explanations, coverage metrics, and future real-user evaluation.
R-05	External language model provides inaccurate advice.	Medium	High	Local deterministic safety layer, controlled fallback, and disclaimers.
R-06	Packaged maps and images make the application too large.	Medium	Medium	Bounded coverage, WebP compression, and release-size inspection.

ID	Risk	Prob.	Impact	Control
R-07	Interaction logging creates privacy concerns.	Medium	High	Consent, minimisation, retention policy, pseudonymous identifiers, and secure deployment.
R-08	Authentication or secret configuration is deployed insecurely.	Low	High	Password hashing, environment secrets, TLS, rotation, and production review.
R-09	Local databases become inconsistent across app upgrades.	Medium	Medium	Schema versioning, migrations, tests, and cautious storage clearing.
R-10	Automated tests differ across SDK or operating-system versions.	Medium	Medium	Record environment, use focused suites, and document known failures.
R-11	Destination images have unclear rights or attribution.	Medium	High	Maintain provenance and licensing records before public release.
R-12	Rural communities are represented inaccurately or without benefit.	Medium	High	Community review, respectful descriptions, corrections, and local partnership.

N.1 Ethical and Responsible-Use Checklist

Consideration	Project response and future obligation
Connectivity equity	Core travel information is packaged locally so lack of mobile data does not remove the entire service.
Community representation	Descriptions should be reviewed with local stakeholders and corrected when they oversimplify culture or accessibility.
Recommendation fairness	Popularity and interaction signals should be monitored for exposure imbalance and should not erase lesser-known destinations.
Privacy	Interaction data should be minimized, consented, protected, retained only as needed, and never silently treated as harmless.
Safety	Route fallbacks and chatbot answers must expose limitations and avoid implying verified emergency or medical authority.
Accessibility	Interfaces should support readable contrast, adequate touch targets, meaningful labels, and continued testing with diverse users.
Image rights	Every distributable image requires provenance, permission or licence compatibility, and appropriate attribution where required.
Environmental impact	Recommendations should avoid encouraging unsafe overcrowding or environmentally damaging access without local guidance.
Transparency	Users should understand when content is local, cached, online-enhanced, generated, approximate, or potentially stale.
Correction process	A future public deployment requires a straightforward way for communities and users to report inaccurate or harmful content.

N.2 Project Milestone Record

Phase	Primary output	Completion evidence	Review focus
Problem definition	Research question, scope, objectives	Proposal and literature review	Relevance and feasibility
Data preparation	Destination and accommodation assets	Validated structured datasets and local media	Coverage and consistency
Architecture	Offline-first design and service boundaries	Diagrams, ADRs, and implementation structure	Failure tolerance
Core Flutter app	Discovery, details, persistence, map	Running screens and local storage	Usability and offline operation
Routing	Cache, OSRM, and Haversine tiers	Routing service and integrated scenarios	Source clarity and safety
Recommender	Semantic, context, collaborative, ranking, explanation	Services, scenario metrics, and tests	Relevance and limitations
Intelligence	Local NLP, retrieval, dialogue, safety, translation	Focused tests and integrated UI	Reliability and safety
Backend	API routes, auth, interactions, analytics	FastAPI implementation and pytest	Contracts and security
Evaluation	Metrics, regression evidence, critical review	Results chapter and appendices	Honest interpretation
Submission	Publication-quality report and reproducible artifacts	Compiled PDF, bibliography, and evidence index	Submission quality

N.3 Submission Evidence Index

Evidence category	Location in report	Assessment value
Research problem and motivation	Introduction and literature review	Establishes significance and academic context
Requirements and scope	Objectives, scope, Appendices E–F	Enables traceable assessment against intended behaviour
Architecture	Design chapter, diagrams, Appendix L	Explains structural decisions and trade-offs
Implementation	Implementation chapter and Appendix M listings	Demonstrates concrete engineering work
Destination and accommodation data	Data sections and Appendix G	Supports catalogue, retrieval, and recommendation claims
Recommendation evaluation	Evaluation chapter and Appendix I	Provides repeatable quantitative quality evidence
Offline verification	Evaluation chapter and test catalogue	Tests the central research contribution
API contracts	Appendix H	Documents client-backend boundaries and failure semantics
User operation	Appendix J	Demonstrates complete intended workflows
Deployment and reproducibility	Appendix K	Supports repeatable setup, build, and troubleshooting
Risk and ethics	Critical evaluation and Appendix N	Demonstrates responsible limitations and future obligations